Red Hat Enterprise Linux for SAP Solutions 9

Deploying SAP HANA Scale-Out System Replication High Availability

Creating an HA cluster for automating scale-out HANA system replication with the \"Advanced Next Generation Interface\" SAP HANA resource agents (\"angi\")

Last Updated: 2026-01-29

Red Hat Enterprise Linux for SAP Solutions 9 Deploying SAP HANA Scale-Out System Replication High Availability

Creating an HA cluster for automating scale-out HANA system replication with the \"Advanced Next Generation Interface\" SAP HANA resource agents (\"angi\")

Legal Notice

Copyright © Red Hat.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at
http://creativecommons.org/licenses/by-sa/3.0/
. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

Java ® is a registered trademark of Oracle and/or its affiliates.

XFS ® is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL ® is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js ® is an official trademark of Joyent. Red Hat Software Collections is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack ® Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

Configure SAP HANA system replication for your scale-out environment and install a Pacemaker cluster. Manage your HANA instance more efficiently by using the RHEL for SAP Solutions components for higher availability.

Table of Contents

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION 4

CHAPTER 1. INTRODUCTION TO SAP HANA SCALE-OUT SYSTEM REPLICATION HA 5
1.1. TERMINOLOGY 5
1.2. PERFORMANCE-OPTIMIZED SAP HANA SCALE-OUT HA 5
1.3. CLUSTER RESOURCE AGENTS AND TOOLS FOR SAP HANA HA 6
1.4. SAP HANA HA/DR PROVIDER HOOKS 7
1.5. SUPPORT POLICIES FOR SAP HANA HIGH AVAILABILITY 8

CHAPTER 2. PLANNING THE HA CLUSTER SETUP 10
2.1. SUBSCRIPTION AND REPOSITORIES FOR SAP HANA HA 10
2.2. OPERATING SYSTEM REQUIREMENTS 10
2.3. STORAGE REQUIREMENTS 11
2.4. NETWORK REQUIREMENTS 12
2.5. HA CLUSTER REQUIREMENTS 12
2.6. SAP HANA PLANNING 14

CHAPTER 3. INSTALLING SAP HANA SCALE-OUT FOR A 4-NODE HA CLUSTER SETUP 16
3.1. MANAGING THE FIREWALLD SERVICE 16
3.1.1. Disabling the firewalld service 16
3.1.2. Configuring the firewalld service for the SAP landscape 17
3.2. CONFIGURING THE HOST NAMES IN /ETC/HOSTS 17
3.3. CONFIGURING THE SHARED SAP FILESYSTEMS 18
3.4. CREATING THE SAP ADMINISTRATIVE USER AND GROUP 19
3.5. CONFIGURING SSH PUBLIC-KEY ACCESS FOR ROOT FOR ALL CLUSTER NODES (OPTIONAL) 20
3.6. INSTALLING A SCALE-OUT SAP HANA INSTANCE 23
3.7. DISABLING SAP HANA INSTANCE AUTOSTART 25
3.8. ADJUSTING THE SAP HANA NAMESERVER CONFIGURATION 26

CHAPTER 4. CONFIGURING THE SAP HANA SYSTEM REPLICATION 28
4.1. PREREQUISITES FOR THE SAP HANA SYSTEM REPLICATION SETUP 28
4.2. PERFORMING AN INITIAL HANA DATABASE BACKUP 28
4.3. CONFIGURING THE PRIMARY HANA REPLICATION SITE 31
4.4. CONFIGURING THE SECONDARY HANA REPLICATION SITE 31
4.5. TESTING THE HANA SYSTEM REPLICATION 35

CHAPTER 5. CONFIGURING THE PACEMAKER CLUSTER 37
5.1. DEPLOYING THE BASIC CLUSTER CONFIGURATION 37
5.2. CONFIGURING GENERAL CLUSTER PROPERTIES 40
5.3. CONFIGURING THE SYSTEMD-BASED SAP STARTUP FRAMEWORK 40
5.4. INSTALLING THE SAP HANA HA COMPONENTS 42
5.5. CONFIGURING THE HANASR HA/DR PROVIDER FOR THE SRCONNECTIONCHANGED() HOOK METHOD 43
5.6. CONFIGURING THE CHKSRV HA/DR PROVIDER FOR THE SRSERVICESTATECHANGED() HOOK METHOD 46
5.7. CREATING THE HANA CLUSTER RESOURCES 47
5.8. CREATING THE VIRTUAL IP RESOURCE 53
5.9. ADDING A SECONDARY (READ-ENABLED) VIRTUAL IP ADDRESS 55
5.10. CONFIGURING ALERT-FENCING OF THE HANA SITE 57

CHAPTER 6. CONFIGURING A QUORUM DEVICE IN THE CLUSTER 59
6.1. CONFIGURING A QDEVICE FOR CLUSTER QUORUM 59
6.1.1. Preparing the quorum device host 59

6.1.2. Configuring a qdevice on a quorum device host 59
6.1.3. Configuring a qdevice in the cluster 60
6.2. CONFIGURING A MAJORITY-MAKER NODE FOR CLUSTER QUORUM 64
6.2.1. Preparing the majority-maker node 64
6.2.2. Updating the host names in /etc/hosts 64
6.2.3. Updating the cluster clone resources 65
6.2.4. Installing the cluster components on the majority-maker node 66
6.2.5. Adding the new node to the cluster 67

CHAPTER 7. TESTING THE SETUP 71
7.1. DETECTING THE SYSTEM REPLICATION STATE CHANGES 71
7.2. TRIGGERING THE INDEXSERVER CRASH RECOVERY 73
7.3. TRIGGERING A HANA TAKEOVER USING CLUSTER COMMANDS 75
7.4. TRIGGERING THE SAPHANAFILESYSTEM FAILURE ACTION 76
7.5. CRASHING THE NODE WITH A PRIMARY INSTANCE 78
7.6. CRASHING THE NODE WITH A SECONDARY INSTANCE 78
7.7. STOPPING THE PRIMARY SITE USING SAP COMMANDS 79
7.8. STOPPING THE SECONDARY SITE USING SAP COMMANDS 80

CHAPTER 8. FINISHING THE SETUP 82
8.1. ENABLING THE AUTOMATIC REGISTRATION OF HANA AFTER A TAKEOVER 82
8.2. REVIEWING THE FINAL CLUSTER STATE 83

CHAPTER 9. MAINTENANCE PROCEDURES 86
9.1. CLEANING UP THE FAILURE HISTORY 86
9.2. TRIGGERING A HANA TAKEOVER USING CLUSTER COMMANDS 86
9.3. UPDATING THE OPERATING SYSTEM AND HA CLUSTER COMPONENTS 86
9.4. PERFORMING MAINTENANCE ON THE SAP HANA INSTANCES 87
9.5. REGISTERING THE FORMER PRIMARY HANA SITE AS A SECONDARY HANA SITE AFTER A TAKEOVER 90

CHAPTER 10. TROUBLESHOOTING 92
10.1. THE SRHOOK CLUSTER ATTRIBUTE VALUE IS INCORRECT 92
10.2. THE HANA INSTANCE DOES NOT START AFTER HOOK CHANGES 93
10.3. A CLUSTER NODE IS REPORTED AS OFFLINE DURING MAINTENANCE 93

APPENDIX A. COMPONENT OPTIONS 95
A.1. HA/DR PROVIDER OPTIONS FOR HANASR 95
A.2. HA/DR PROVIDER OPTIONS FOR CHKSRV 95
A.3. SAPHANATOPOLOGY RESOURCE PARAMETERS 96
A.4. SAPHANACONTROLLER RESOURCE PARAMETERS 97
A.5. SAPHANAFILESYSTEM RESOURCE PARAMETERS 102

APPENDIX B. USEFUL INFORMATION 103
B.1. EXPLAINING CLUSTER INFORMATION 103

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION

We appreciate your feedback on our documentation. Let us know how we can improve it.

Submitting feedback through Jira (account required)

1. Make sure you are logged in to the Jira website.
2. Click on this link to provide feedback.
3. Enter a descriptive title in the **Summary** field.
4. Enter your suggestion for improvement in the **Description** field. Include links to the relevant parts of the documentation.
5. Click **Create** at the bottom of the dialogue.

CHAPTER 1. INTRODUCTION TO SAP HANA SCALE-OUT SYSTEM REPLICATION HA

Configuring the SAP HANA system replication between two identical HANA sites enables a basic resiliency of the database. You can configure these two sites in a Pacemaker cluster for advanced high availability that automatically handles the service recovery in the case of a failure on the primary instance side.

1.1. TERMINOLOGY

- **node**
  One host or system in a HA cluster setup, also called a cluster member.
- **cluster**
  Cluster is the high-availability setup using the Pacemaker cluster manager from the RHEL HA Add-On. It consists of two or more members, or nodes.
- **instance**
  One set of SAP HANA systems that belong to one HANA site. In single-host (scale-up) HANA environments, one HANA site consists of a single HANA instance. In multiple-host (scale-out) HANA configurations, each HANA site consists of two or more HANA instances.
- **primary**
  The primary HANA instance or primary site refers to the instance which is the active HANA instance or site. In single-host setups (scale-up), this is one system. In multiple-host (scale-out) setups, the primary database stretches across multiple systems of one HANA site and the systems have different roles in the HANA environment to distribute load.
- **secondary**
  The secondary HANA instance or secondary site refers to the SAP HANA instance or site which is configured to be synced with the primary HANA instance through the SAP HANA system replication mechanism. This instance preloads the in-memory data of the primary instance and is ready to take over if the primary instance fails.

1.2. PERFORMANCE-OPTIMIZED SAP HANA SCALE-OUT HA

Performance-optimized means that there is only a single SAP HANA instance running on each node that has control over most of the resources, such as CPU and RAM, on each node. This means that the SAP HANA instances can run with as much performance as possible.

You configure the HANA environment without HANA standby hosts and only one coordinator name server per replication site. This master name server controls the landscape of each site. The HANA host auto-failover functionality using idle standby hosts is not necessary, because the Pacemaker cluster controls the high availability of the HANA database and manages the HANA system replication.

With a performance-optimized SAP HANA system replication setup of SAP HANA 2.0 SPS1 or newer you can also configure read access to the secondary system to reduce the load on the primary instance. For more information see the SAP documentation for Active/Active (Read Enabled) configuration.

Additional resources

- Supported Scenarios for Automated SAP HANA Scale-Out System Replication
- Configuration guides for the setup on public cloud environments

1.3. CLUSTER RESOURCE AGENTS AND TOOLS FOR SAP HANA HA

The high-availability (HA) cluster configuration for managing SAP HANA system replication setups works with multiple resource agents and other tools that combine their functionality for the expected behavior. The Advanced Next Generation Interface ("angi") resource agents are identical for scale-up and scale-out environments. In upstream they are also called **SAPHanaSR-angi**.

On RHEL this generation of combined resource agents and tools is provided in the package **sap-hana-ha**.

- SAPHanaTopology
  The **SAPHanaTopology** resource agent gets status information from the SAP HANA environment and saves it to cluster properties. The agent also starts and monitors the local **SAP HostAgent**, which is required for starting, stopping and monitoring the HANA instances. A configuration process in SAP HANA called system replication hook adds replication health information as well to the saved properties. Based on the collected environment data, the resource agent defines a dedicated health score of the cluster node. This scoring is used by the cluster to decide if it must initiate the switch of the system replication from one site to the other.

- SAPHanaController
  The **SAPHanaController** resource agent monitors and manages the SAP HANA environment. In case of a failure of the HANA instance, the resource determines which recovery action it takes and executes the commands for an automatic switch, or it changes the active site of the system replication.

- SAPHanaFilesystem
  The **SAPHanaFilesystem** resource agent monitors mounted SAP HANA filesystems for read/write access. It does not mount or unmount filesystems but decides on actions based on HANA system replication status if a monitor fails, allowing for faster takeover actions. On primary HANA sites, if a monitor fails, the cluster tries to stop and restart the resource first. If that fails and HANA system replication is in sync, the cluster fences the node. If the HANA system replication is not in sync, the cluster repeats the restart until it is successful or a migration threshold is reached. On secondary HANA sites, the cluster is not aware of monitor failures. This resource agent is particularly useful for HANA scale-out systems that rely on NFS shares for **/hana/shared/<SID>/**, which, if failed, can stop HANA without timely cluster action. However, you can also use the resource agent with local filesystems on scale-up systems.

- SAPHanaSR-showAttr
  The **SAPHanaSR-showAttr** tool shows cluster attributes for the SAP HANA system replication automation in a preformatted overview including the HANA topology that shows whether it is a scale-up or scale-out environment. The default output includes the system replication status between the nodes and other related status information. The script retrieves the information from the Cluster Information Base (CIB), where other resource agents or hook scripts store updates during their regular checks or from HANA events, respectively. Due to this, the information can contain outdated states until it is updated again. Use HANA tools to get real-time status information from the landscape.

- SAPHanaSR-hookHelper
  The **SAPHanaSR-hookHelper** tool is a helper script that is used by other SAP HANA HA components for specific shared functionality, such as certain fencing features.

- SAPHanaSR-alert-fencing
  The **SAPHanaSR-alert-fencing** script is a cluster alert agent. In scale-out setups with HANA HA, you can configure this alert agent to trigger the cluster to fence all nodes of the same HANA

site after one node of that site is fenced due to a failure. This functionality only applies to scale-out configurations and has no effect in a scale-up setup.

NOTE

Verify that the new generation of resource agents is available for your RHEL version. Check Minimum supported package versions for SAP HANA Scale-Up and Scale-Out System Replication HA solutions.

Additional resources

- Appendix A. Component options
- SAP Note 2063657 - SAP HANA System Replication Takeover Decision Guideline
- **ocf_heartbeat_SAPHanaTopology(7)**, **ocf_heartbeat_SAPHanaController(7)** and **ocf_heartbeat_SAPHanaFilesystem(7)** man pages on your system

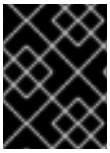
1.4. SAP HANA HA/DR PROVIDER HOOKS

Current versions of SAP HANA provide an API in the form of hooks that allow the HANA instance to send notifications for certain events, for example the loss or establishment of the system replication. For each event, the HANA instance calls the configured hooks, also called HA/DR providers. Hooks are custom Python scripts which process the events that HANA sends and the scripts can trigger different actions based on the event information.

You must add the HA/DR provider definition to the HANA global configuration to enable the required functionality of triggering additional actions for certain events.

HanaSR for the srConnectionChanged() hook method

The **HanaSR** hook is required for processing the **srConnectionChanged()** hook method. This method is used by the primary HANA instance for a notification of any change in the HANA system replication status. The primary HANA instance calls the **HanaSR** HA/DR provider when a HANA system replication related event occurs. The hook script **HanaSR.py** then parses **srConnectionChanged()** events for the system replication status detail and as a result it updates the **srHook** cluster attribute. This attribute is used by the resource agents to evaluate the landscape health and make decisions. The value of the system replication or sync state defines if the cluster recovers a failed primary instance on the same node or if it triggers a takeover to the secondary. The takeover is only triggered when the system replication is fully in sync, which means the HANA data is consistent between the HANA sites.



IMPORTANT

You must configure the **HanaSR** hook to enable the **srConnectionChanged()** hook method for proper function and full support of the HA cluster setup.

ChkSrv for the srServiceStateChanged() hook method

When the HANA instance detects an issue with a HANA indexserver process it recovers from the problem by stopping and restarting the **hdbindexserver** service automatically through an internal mechanism.

However, especially for very large HANA instances, the **hdbindexserver** service can take a very long time for the stopping phase of this recovery process. Although HANA reports this service degradation

not as an error in the HANA landscape, the situation poses a risk to the data consistency if anything else fails in the instance during that time. To improve the unpredictable service recovery time, you can configure the **ChkSrv** hook to stop or kill the entire affected HANA instance instead.

In a setup with automatic failover enabled (**PREFER_SITE_TAKEOVER=true**), the instance stop leads to a takeover if the secondary node is in a healthy state. Otherwise, instance recovery happens locally, but the enforced local instance restart accelerates the process.

The HANA instance calls the **ChkSrv** hook when an event occurs. The hook script **ChkSrv.py** processes the **srServiceStateChanged()** hook method and executes actions based on the results of the filters it applies to event details. This way the **ChkSrv.py** hook script can distinguish a HANA **hdbindexserver** process that is being stopped and restarted by HANA after a failure from the same process being stopped as part of an intended instance shutdown. When the hook script determines that the event is caused by a failure it triggers the configured action.

The **ChkSrv.py** hook script has multiple options to define what happens when an indexserver failure event is detected:

- ignore
  This action just writes the parsed events and decision information to a dedicated logfile. This is useful for testing and verifying what the hook script would do when activating **stop** or **kill** actions.

- stop
  This action executes a graceful **StopSystem** for the instance through the **sapcontrol** command.

- kill
  This action executes the **HDB kill-<signal>** command with a default signal 9, which can be configured. The result is the same as when using **stop**, but can be faster.

- fence
  This action triggers the cluster to **fence** the node on which the indexserver failed. This option uses the **SAPHanaSR-hookHelper** tool and requires a sudo entry for the tool for the **<sid>adm** user.

NOTE

Any indexserver failure is treated individually by HANA. The same processes are always triggered for every single indexserver issue.

Enabling the **srServiceStateChanged()** hook is optional.

Additional resources

- Appendix A. Component options

1.5. SUPPORT POLICIES FOR SAP HANA HIGH AVAILABILITY

Red Hat supports the following components of the solution:

- Basic operating system configuration for running SAP HANA on RHEL, based on SAP guidelines

- RHEL HA Add-On

- Red Hat HA solutions for SAP HANA system replication

Additional resources

- Support Policies for RHEL High Availability Clusters - Management of SAP HANA in a Cluster
- SAP Note 3108302 - SAP HANA DB: Recommended OS Settings for RHEL 9

CHAPTER 2. PLANNING THE HA CLUSTER SETUP

Plan your setup carefully to ensure that all requirements for the HA cluster configuration for automating the HANA system replication of your HANA landscape are met.

2.1. SUBSCRIPTION AND REPOSITORIES FOR SAP HANA HA

The solutions for SAP HANA in a Pacemaker cluster for High Availability (HA) are provided in dedicated repositories. The RHEL for SAP Solutions subscription is required to access all relevant content. In addition to the standard RHEL repos the subscription provides access to the following repos, which are required to set up the SAP HANA HA solution:

- **High Availability**
  The RHEL HA Add-On's content is stored in a repository named High Availability. The repository ID is represented as **rhel-9-for-<arch>-highavailability-e4s-rpms**.

- **SAP Solutions**
  Name of the repository that contains the SAP HANA specific content. The repository ID is represented as **rhel-9-for-<arch>-sap-solutions-e4s-rpms**.

The **<arch>** denotes the specific hardware architecture:

- **x86_64**

- **ppc64le**

Example list of repositories enabled as part of the RHEL for SAP Solutions subscription:

```
[root]# dnf repolist
Updating Subscription Management repositories.
repo id                                  repo name
rhel-9-for-x86_64-appstream-e4s-rpms       Red Hat Enterprise Linux 9 for x86_64 - AppStream -
Update Services for SAP Solutions (RPMs)
rhel-9-for-x86_64-baseos-e4s-rpms          Red Hat Enterprise Linux 9 for x86_64 - BaseOS - Update
Services for SAP Solutions (RPMs)
rhel-9-for-x86_64-highavailability-e4s-rpms Red Hat Enterprise Linux 9 for x86_64 - High Availability -
Update Services for SAP Solutions (RPMs)
rhel-9-for-x86_64-sap-netweaver-e4s-rpms    Red Hat Enterprise Linux 9 for x86_64 - SAP
NetWeaver - Update Services for SAP Solutions (RPMs)
rhel-9-for-x86_64-sap-solutions-e4s-rpms    Red Hat Enterprise Linux 9 for x86_64 - SAP Solutions -
Update Services for SAP Solutions (RPMs)
```

Additional resources

- Enabling required repositories

- Overview of Red Hat Enterprise Linux for SAP Solutions Subscription

- RHEL for SAP Subscriptions and Repositories

2.2. OPERATING SYSTEM REQUIREMENTS

Deploy your host operating system as described in Installing RHEL 9 for SAP Solutions .

Follow SAP Note 3108302 - SAP HANA DB: Recommended OS Settings for RHEL 9 to configure architecture specific settings, kernel parameters and check the minimum required Linux kernel and HANA versions.

Apply the operating system post-installation configuration for SAP HANA hosts as described in SAP Note 3108316 - Red Hat Enterprise Linux 9.x: Installation and Configuration .

Root privileges

For the HANA installation and the cluster HA setup the **root** user or a privileged user that can run any sudo commands is required.

Additional resources

sudo(8) man page on your system

2.3. STORAGE REQUIREMENTS

You can find information about Sizing SAP HANA in the SAP HANA Master Guide.

There is no communication between both scale-out environments on the storage level. As a result, you must complete the storage configuration on each scale-out environment before installing the HANA instances.

For the setup of a scale-out SAP HANA environment with HANA system replication between two HANA sites you can configure the storage as shared or non-shared.

Shared storage

Configure 3 NFS shares for the mountpoints **/hana/data**, **/hana/log**, **/hana/shared**. For each HANA site you must create a dedicated set of the shares.

Example NFS storage details for the HANA instance in datacenter 1:

Method	NFS Server	NFS Path	Mount Point
NFS	**nfs01-datacenter1a.example.com**	**/dc1/data**	**/hana/data**
NFS	**nfs01-datacenter1a.example.com**	**/dc1/log**	**/hana/log**
NFS	**nfs01-datacenter1a.example.com**	**/dc1/shared**	**/hana/shared**

Example NFS storage details for the HANA instance in datacenter 2:

Method	NFS Server	NFS Path	Mount Point
NFS	**nfs01-datacenter2a.example.com**	**/dc2/data**	**/hana/data**
NFS	**nfs01-datacenter2a.example.com**	**/dc2/log**	**/hana/log**
NFS	**nfs01-datacenter2a.example.com**	**/dc2/shared**	**/hana/shared**

Non-shared storage

A non-shared storage configuration requires the integration of the storage connector. The storage connector manages access to the LUNs or LVM devices over SCSI or LVM locking mechanisms. You can only use this method for **/hana/data** and **/hana/log**.

Use the SAP HANA Fiber Channel Storage Connector Admin Guide for the setup of non-shared storage.

For **/hana/shared** you must configure an NFS share for each instance, even if you use non-shared storage for **/hana/data** and **/hana/log**.

Additional resources

- Multiple-Host System Concepts
- SAP Note 1900823 - SAP HANA Storage Connector API

2.4. NETWORK REQUIREMENTS

You can find information about SAP HANA network architecture considerations in the SAP HANA Administration Guide.

For the SAP HANA system replication setup in a HA cluster we recommend that you configure dedicated networks and connections for the cluster communication traffic, which is separate from any HANA network traffic.

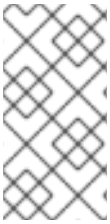
Additional resources

- SAP HANA Master Guide
- Network Configuration for SAP HANA System Replication

2.5. HA CLUSTER REQUIREMENTS

Fencing

For a supported HA cluster setup using the RHEL HA Add-on you must configure a fencing or STONITH device on each cluster node. Which fencing or STONITH device you can use depends on the platform the cluster is running on. Check the Support Policies for RHEL High Availability Clusters - Fencing/STONITH for recommendations on fencing agents or consult your hardware or cloud provider to find out which fence device is supported on their platform.



NOTE

fence_scsi or **fence_mpath** as fencing/STONITH mechanism requires shared storage between the cluster nodes that is fully managed by the HA cluster. If your SAP environment does not include such a shared disks setup, using these fencing options is not supported.

Additional resources

- High Availability Add-On concepts - Fencing
- Support Policies for RHEL High Availability Clusters - General Requirements for Fencing/STONITH

Quorum

HANA environments with HANA system replication managed by a Pacemaker cluster in a performance-optimized setup always consist of an even number of HANA nodes. Each cluster member automatically counts as one vote in the quorum calculations, which the cluster uses to decide which nodes can continue running when there is a communication disruption between the nodes. An even number of cluster nodes can lead to a 50/50 split and there is a risk of a split-brain situation, where both partitions continue running and cause conflicts or data corruption in the running services.

We highly recommend that you configure an additional quorum vote in the cluster to have an odd number of votes and improve the availability of the service even during multiple cluster interconnect failures. See Exploring Concepts of RHEL High Availability Clusters - Quorum for more details about the concept and its benefits.

You can use a qdevice or an additional cluster node to add a quorum vote to your cluster. Both methods require a separate host and have different advantages and limitations that you must consider.

See Configuring a quorum device in the cluster for the configuration steps of the following quorum device methods:

- **qdevice**
  - You must configure a dedicated host that is not a member of any cluster.
  - Ideally you place the qdevice host in a different location or availability zone than any cluster members.
  - You can configure no more than one qdevice per cluster.
  - A single qdevice host can serve qdevices to multiple different clusters. You do not need additional qdevice hosts if your different clusters can reach the same qdevice host.
  - The qdevice is visible in the quorum configuration only and does not require any change or considerations with the cluster resource settings.
  - The qdevice communicates through the network. Use any production network, for example, the HANA client network.

  - Ideally you configure a highly available network connection, for example, bonded interfaces on the qdevice host.

- **majority-maker node**
  - You must configure a dedicated host that becomes a member of the cluster you want to use it for.
  - Ideally you place the majority-maker host in a different location or availability zone than other cluster members.
  - The majority-maker host can only be a member of one cluster. You must configure a separate host for the same functionality in any other cluster.
  - The communication with this host happens through corosync, like any other cluster member.
  - You must adjust your cluster resource settings and add cluster constraints to prevent the cluster from running any resources on this node and leave it out of node target calculations for cloned resources.
  - You can configure multiple additional cluster nodes that are restricted to only serve as quorum votes.
  - Ideally you configure a highly available network connection, for example, bonded interfaces on the majority-maker host.

Additional resources

- High Availability Add-On concepts - Quorum
- Configuring quorum devices
- Exploring Concepts of RHEL High Availability Clusters - Quorum
- Design Guidance for RHEL High Availability Clusters - Considerations with qdevice Quorum Arbitration

2.6. SAP HANA PLANNING

The **sap-hana-ha** package provides the combined resource agents which support setups with SAP HANA 2.0 SPS05 rev 59.04 or newer.

To prepare the HANA setup you can define a list of parameters that you require for the installation and configuration of the planned environment.

Example SAP HANA configuration parameters are show below:

Parameter	Example value
HANA SID	**RH1**
HANA instance number	**02**
HANA site 1 name	**DC1**

Parameter	Example value
HANA site 2 name	**DC2**
HANA site 1 node 1 FQDN	**dc1hana1.example.com**
HANA site 1 node 2 FQDN	**dc1hana2.example.com**
HANA site 2 node 1 FQDN	**dc2hana1.example.com**
HANA site 2 node 2 FQDN	**dc2hana2.example.com**
HANA DB 'SYSTEM' user password	**<HANA_SYSTEM_PASSWORD>**
SAP system group ID	**10001**
SAP system group name	**sapsys**
SAP local administrator user ID	**10200**
SAP local administrator user name	**sapadm**
HANA administrative user ID	**10210**
HANA administrative user name	**rh1adm**

Additional resources

- SAP HANA Server Installation and Update Guide
- Software Download
- SAP Note 2057595 - FAQ: SAP HANA High Availability
- SAP Note 3007062 - FAQ: SAP HANA & Third Party Cluster Solutions

CHAPTER 3. INSTALLING SAP HANA SCALE-OUT FOR A 4-NODE HA CLUSTER SETUP

The examples in the following configuration steps demonstrate the setup on 2 scale-out nodes per HANA site, which results in an installation of 4 HANA nodes.

You can apply the same steps to more scale-out nodes per site, resulting in 6 or more nodes in total. Each HANA site must consist of the same amount of identically configured nodes.

3.1. MANAGING THE FIREWALLD SERVICE

On RHEL the **firewalld** systemd service is enabled by default when installed and starts with a basic configuration.

For your planned SAP landscape you must decide if you want to manage all port and connection requirements in the firewall service on each cluster node, or if this is handled separately in the security design of your network infrastructure. You must disable the **firewalld** service in the case that you do not need to manage a firewall on the operating system level on each cluster node. If the local firewall service remains running without the necessary port configuration, it blocks the cluster communication and the connections between your SAP systems.

For your SAP landscape and HA setup to work you must implement one of the following options:

- Disabling the firewalld service
- Configuring the firewalld service for the SAP landscape

3.1.1. Disabling the firewalld service

The **firewalld** service is installed and enabled by default as part of the "Server" package group. You must disable it if you do not use it in your network security strategy.

Prerequisites

- You are managing firewall rules outside of the individual host operating systems as part of your security concept.

Procedure

- Stop and disable the **firewalld** service on each cluster node. The **--now** parameter automatically stops the disabled service. Run this on each system of your planned landscape:

```
[root]# systemctl disable --now firewalld.service
```

Verification

- Verify that the firewalld service is disabled on each node.

```
[root]# systemctl status firewalld.service
○ firewalld.service - firewalld - dynamic firewall daemon
    Loaded: loaded (/usr/lib/systemd/system/firewalld.service; disabled; preset: enabled)
    Active: inactive (dead)
      Docs: man:firewalld(1)
```

Additional resources

systemctl(1) man page on your system

3.1.2. Configuring the firewalld service for the SAP landscape

Check the SAP documentation for the Ports and Connections that you have to enable in the firewall for your SAP landscape. Consider all SAP components in your setup that require incoming or outgoing communication and connections between the different hosts in your landscape.

Configure the **firewalld** service on each of your SAP hosts using the methods that fit your requirements best. Consult Configuring firewalls and packet filters for the details on how to use the **firewalld** service effectively.

Additional resources

- Landscape Management and Network Administration
- **firewalld(1)** and **firewall-cmd(1)** man pages on your system

3.2. CONFIGURING THE HOST NAMES IN /ETC/HOSTS

For a consistent host name resolution between all systems in your HANA and HA setup we recommend adding them to the **/etc/hosts** file on each node.

If you configure the HANA Internal Host Name Resolution you must ensure that the **/etc/hosts** entries for the same host names are consistent with the HANA configuration.

Procedure

- Add the host names of all hosts to the **/etc/hosts** on all cluster nodes:

```
[root]# cat /etc/hosts
...
192.168.100.101 dc1hana1.example.com dc1hana1
192.168.100.102 dc1hana2.example.com dc1hana2
192.168.100.103 dc2hana1.example.com dc2hana1
192.168.100.104 dc2hana2.example.com dc2hana2
```

Verification

- Check that you can ping the hosts. This step is optional and an example only for a basic verification. The system resolves entries in **/etc/hosts** when you use the ping command:

```
[root]# ping dc1hana2.example.com
PING dc1hana2.example.com (192.168.100.102) 56(84) bytes of data.
64 bytes from dc1hana2.example.com (192.168.100.102): icmp_seq=1 ttl=64 time=0.017 ms
…
```

Additional resources

- How should the /etc/hosts file be set up on RHEL cluster nodes?

- Host Name Resolution for System Replication
- **hosts(5)** man page on your system

3.3. CONFIGURING THE SHARED SAP FILESYSTEMS

You must configure the shared filesystems on all systems of each HANA site they belong to.

Prerequisites

- You have prepared the shared NFS-based filesystems, and all cluster nodes of each HANA site are able to access their related shares. **The NFS shares must be external and not exported on one of the cluster nodes.**

Procedure

1. Create the directories for the shared filesystems:

```
[root]# mkdir -p /hana/{shared,data,log}
```

2. Add the shared NFS filesystems to **/etc/fstab** to mount them automatically on system start. Configure the mount options that apply to your environment. The following is a basic example:

```
[root]# vi /etc/fstab
…
<nfs_server>:/<site_path>/data /hana/data nfs4 defaults 0 0
<nfs_server>:/<site_path>/log /hana/log nfs4 defaults 0 0
<nfs_server>:/<site_path>/shared /hana/shared nfs4 defaults 0 0
```

   - Replace **<nfs_server>** with the NFS server DNS name or the IP address of each share, for example, **nfs01-datacenter1a.example.com**.
   - Replace **<site_path>** with the site specific root path, for example, **dc1** on the nodes of one HANA site.

3. Reload the **systemctl** daemon to make the new **/etc/fstab** entries known to systemd:

```
[root]# systemctl daemon-reload
```

4. Mount any new filesystems that you configured in the **/etc/fstab**:

```
[root]# mount -a
```

5. Repeat the configuration steps on each cluster node.

Verification

1. Check that the filesystems are mounted, for example, on HANA site 1:

```
[root]# df -hP | grep hana
nfs01-datacenter1a.example.com:/dc1/hana/data    8.0E   32G  8.0E   1% /hana/data
nfs01-datacenter1a.example.com:/dc1/hana/log     8.0E   32G  8.0E   1% /hana/log
nfs01-datacenter1a.example.com:/dc1/hana/shared  8.0E   32G  8.0E   1% /hana/shared
```

2. Check that the systemd mount targets exist for the filesystems configured in the **/etc/fstab**:

```
[root]# systemctl list-units --all | grep -e 'hana*.*mount' | column -t
hana-data.mount    loaded  active  mounted  /hana/data
hana-log.mount     loaded  active  mounted  /hana/log
hana-shared.mount  loaded  active  mounted  /hana/shared
```

3. Repeat the verification steps on each cluster node.

Additional resources

- Persistently mounting file systems
- **mkdir(1)**, **fstab(5)**, **mount(8)** and **systemctl(1)** man pages on your system

3.4. CREATING THE SAP ADMINISTRATIVE USER AND GROUP

In a high-availability environment where the highly available service can move between different systems using shared storage, you must configure the service's user and groups with identical numerical values for their user ID (UID) and group ID (GID). Different IDs for the same service users or groups cause access conflicts and prevent you from switching the service between the cluster nodes.

Prepare the following operating system group:

- **sapsys**

Prepare the following operating system users:

- **sapadm**
- **<sid>adm**, using your target SID

Prerequisites

- You have reserved identical user and group IDs for the required groups and users, for example, in your central identity management system for service users.

Procedure

1. Create the **sapsys** group. Use the prepared group ID, for example, ID **10001**:

```
[root]# groupadd -g 10001 sapsys
```

2. Create the **sapadm** user as a member of the **sapsys** group. The user does not need a login shell. Use the prepared user ID, for example ID, **10200**:

```
[root]# useradd -u 10200 -g sapsys sapadm \
-c 'SAP Local Administrator' -s /sbin/nologin
```

3. Create the **<sid>adm** user as a member of the **sapsys** group. Use the prepared user ID, for example, ID **10210** for user **rh1adm**:

```
[root]# useradd -u 10210 -g sapsys rh1adm \
-c 'SAP HANA Administrator' -s /bin/sh
```

As the user shell, we recommend that you either use **/bin/sh** or **/bin/csh**. SAP installations provide user profiles and useful shell aliases in these shells.

4. Repeat the steps on all nodes.

Verification

1. Check that the users **sapadm** and **<sid>adm** exist and have the correct groups and IDs configured, for example:

```
[root]# id sapadm rh1adm
uid=10200(sapadm) gid=10001(sapsys) groups=10001(sapsys)
uid=10210(rh1adm) gid=10001(sapsys) groups=10001(sapsys)
```

2. Check that the users have the correct description, home directory and shell defined:

```
[root]# grep -E 'sapadm|rh1adm' /etc/passwd
sapadm:x:10200:10001:SAP Local Administrator:/home/sapadm:/sbin/nologin
rh1adm:x:10210:10001:SAP HANA Administrator:/home/rh1adm:/bin/sh
```

3. Repeat the check on all nodes and verify that the names and IDs are identical.

Additional resources

- Creating Operating System Users and Groups
- **groupadd(8)** and **useradd(8)** man pages on your system

3.5. CONFIGURING SSH PUBLIC-KEY ACCESS FOR ROOT FOR ALL CLUSTER NODES (OPTIONAL)

There are steps in the configuration in which you potentially require passwordless root access to the cluster nodes . This can be achieved by setting up SSH public-key authentication between different servers. If this can be used depends on your specific HANA setup and the security policies of your company.

Passwordless root access might be needed in the following situations:

- Accessing all hosts of the same HANA site during the database installation. This applies when your HANA site consists of more than one node like in a scale-out setup.
- Accessing the primary site from the secondary site for the HANA system replication configuration.

Procedure

1. Generate an ssh key pair. When no key type is defined, it creates an RSA key by default, like in the following example for the **root** user:

```
[root]# ssh-keygen
```

2. Option 1, if you have ssh **PasswordAuthentication** enabled on the remote system: Use the **ssh-copy-id** tool to add the ssh public key to the **authorized_keys** on the remote system. This automatically creates the **.ssh**/ directory and **authorized_keys** file with correct permissions for the target user on the remote system. Run it on the host on which you created the ssh key in the previous step and enter the target user password when prompted:

```
[root]# ssh-copy-id <remote_system>
```

   In the case of the **root** user, this only works if the ssh config allows **PermitRootLogin** and you can provide the **root** user password in the prompt. Check the ssh configuration setting on the remote system if you face access permission issues even after you have enabled **PasswordAuthentication**. Consult your security policies before you enable these parameters on your HANA systems.

3. Option 2, if password login to the target user on the remote host is prohibited or otherwise not possible: Configure the ssh key access on the remote system manually.

   a. Create the **.ssh**/ directory in the target user's home path on the remote system, if it does not exist yet. Run this on the remote system, for example, for the **root** user:

```
[root]# mkdir /root/.ssh
```

   b. Change the permissions of the new **.ssh**/ directory. For security reasons the ssh key access does not work when the permissions are not correct. Run this on the remote system:

```
[root]# chmod 0700 /root/.ssh
```

   c. Copy the ssh public key from the .pub file that was created by the previous **ssh-keygen**, for example, **id_rsa.pub** in the default setting:

```
[root]# cat /root/.ssh/id_rsa.pub
ssh-rsa … root@<node1>
```

   d. Add the public key to the **authorized_keys** file. The command creates the file if it does not exist yet, otherwise it appends the key to the existing content. Run this on the remote system, for example, on **dc1hana2**:

```
[root]# cat << EOF >> /root/.ssh/authorized_keys
ssh-rsa … root@<node1>
EOF
```

   e. Ensure that the **authorized_keys** file has the correct permissions, otherwise the ssh key access is blocked for security reasons:

```
[root]# chmod 0600 /root/.ssh/authorized_keys
```

4. Access each system and log in from any source host to any remote host that you require for the setup. On first login you must accept each new connection once in an interactive prompt. This saves each host and key in the ssh **known_hosts** file by default.

   a. Option 1: Log in from each host to each other host and accept the key fingerprint once to save it to the local **known_hosts** file. Subsequent logins to the same host will not require further interaction, unless the key changes. This is a security measure to prevent unsolicited

changes of the ssh keys. The following example confirms the authenticity of host **dc1hana2**:

```
[root]# ssh dc1hana2
The authenticity of host 'dc1hana2 (***)' can't be established.
ED25519 key fingerprint is SHA256:**********************************.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
…
```

b. Option 2: If you configure ssh key access between multiple systems you can use **ssh-keyscan** to collect the public host key from multiple hosts and save it to the local **known_hosts** file in a single step per host. Run this on each system for which you distributed the public key and list all remote hosts that you potentially access from this node and user, for example, for the **root** user on host **dc1hana1**:

```
[root]# ssh-keyscan -t rsa -f - >> /root/.ssh/known_hosts
dc1hana2
dc2hana1
dc2hana2
<Ctrl-d>
# dc1hana2:22 SSH-2.0-OpenSSH_8.7
# dc2hana1:22 SSH-2.0-OpenSSH_8.7
# dc2hana2:22 SSH-2.0-OpenSSH_8.7
```

- Limit the scan to the type of key you created, for example, the default RSA. When no type is provided the scan checks for multiple types, which adds unnecessary output.
- **-f -** allows you to provide a list of hosts on the standard input. Instead of the **-** you can use a file, which you prepare upfront with the list of hosts. You can also enter a single hostname instead of the **-f** parameter to collect the key of one host at a time.
- In the case of the standard input list you end the input with **Ctrl** and **d**.
- The **>>** shell redirection after the scan command directly appends the collected keys to the **known_hosts** file. If the file does not exist yet it is created in the process.

Verification

- Check the **known_hosts** entries, for example, on **dc1hana1**:

```
[root]# cat /root/.ssh/known_hosts
dc1hana2 ssh-rsa ...
dc2hana1 ssh-rsa ...
dc2hana2 ssh-rsa ...
```

- **Test the access from each source system to every remote system and ensure that every connection direction that you possibly need works without interactive prompts:**

```
[root]# ssh <remote_system>
```

Additional resources

- Using secure communications between two systems with OpenSSH
- **ssh-keygen(1)**, **ssh_config(5)**, **ssh-copy-id(1)**, **ssh-keyscan(1)** and **ssh(1)** man pages on your system

3.6. INSTALLING A SCALE-OUT SAP HANA INSTANCE

A HANA scale-out configuration consists of at least 2 HANA instances per system replication site.

Install the HANA instances with the same SID and instance number on all nodes. The setup of the system replication sites must be identical.

The following installation steps are an example of an interactive installation using the command-line interface. Check the SAP HANA Server Installation and Update Guidefor more information about installation options and other details.

Prerequisites

- You have installed and configured RHEL 9 on all cluster nodes according to the Operating system requirements.
- You have prepared the details for your HANA instances, see SAP HANA planning.
- You have followed the SAP software download guides in Software Download, downloaded the SAP HANA installation media from the SAP Software Download Centerand the media is available on each node.
- You have verified that you can resolve the host names of the additional nodes of one site from the main node of the site.
- You have verified that you can connect to the additional nodes of one site from the main node using the root user and ssh.
- You have configured a time synchronization service on all nodes. See Configuring time synchronization for details.

Procedure

1. Go to the directory which contains the installation media, for example, **/sapmedia/hana**:

```
[root]# cd /sapmedia/hana
```

2. Unpack the installation media:

```
[root]# unzip <sap_hana_software>.ZIP
```

3. Go into the path of the unpacked installation media:

```
[root]# cd /sapmedia/hana/DATA_UNITS/HDB_LCM_LINUX_<arch>
```

4. Run the SAP HANA Lifecycle Management tool (HDBLCM) for an interactive installation:

```
[root]# ./hdblcm
```

In the interactive mode the installer asks you for all the required information, including the System ID (SID), Installation Number (instance), the filesystem location of data and log volumes, and more.

In a scale-out installation you run the installer on the main node of one HANA site and provide any additional nodes of the same site as an installation parameter. For example, you run the installer for site 1 on **dc1hana1** and add node**dc1hana2** as an additional host name to add when the prompt asks for it.

Optionally you can use the batch mode of the command-line installation tool and provide your configuration parameters in one step. For more details see Use Batch Mode to Perform Platform LCM Tasks in the SAP HANA Server Installation and Update Guide.

5. Repeat all steps on the main node of the second site. For the HANA system replication to work you must ensure that each HANA site consists of the same amount of systems with an identical HANA configuration.

Verification

1. Switch to the **<sid>adm** user:

```
[root]# su - rh1adm
```

2. Check the HANA instance runtime information as user **<sid>adm**:

```
rh1adm $ HDB info
USER          PID     PPID  %CPU        VSZ        RSS COMMAND
rh1adm      46483    46482   0.0       9952       5248 -sh
rh1adm      46679    46483   0.0       7524       3968  \_ /bin/sh /usr/sap/RH1/HDB02/HDB info
rh1adm      46708    46679   0.0       9972       3584      \_ ps fx -U rh1adm -o
user:8,pid:8,ppid:8,pcpu:5,vsz:10,rss:10,args
rh1adm      45538        1   0.0     566868      40816 hdbrsutil  --start --port 30003 --volume 3
…
rh1adm      44802        1   0.0     566792      40872 hdbrsutil  --start --port 30001 --volume 1
…
rh1adm      44689        1   0.0       9308       3164 sapstart
pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_dc1hana1
rh1adm      44698    44689   0.1     455136      89788  \_
/usr/sap/RH1/HDB02/dc1hana1/trace/hdb.sapRH1_HDB02 -d -nw -f
/usr/sap/RH1/HDB02/dc1hana1/daemon.ini
pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_dc1hana1
rh1adm      44724    44698  65.8   14409956   10479592      \_ hdbnameserver
rh1adm      45361    44698   0.7    1470088     199224      \_ hdbcompileserver
rh1adm      45364    44698  79.6    3507192    2715988      \_ hdbpreprocessor
rh1adm      45406    44698  74.4    9271804    5685896      \_ hdbindexserver -port 30003
rh1adm      45409    44698   4.6    5078284    1361068      \_ hdbxsengine -port 30007
rh1adm      45964    44698   2.1    2777544     457496      \_ hdbwebdispatcher
rh1adm      44450        1   0.5     499960      58004 /usr/sap/RH1/HDB02/exe/sapstartsrv
pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_dc1hana1
```

3. Verify as **<sid>adm** on all sites that the HANA instances are running on all nodes in the site and their status is **GREEN** in the instance list, for example, on site 1:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function GetSystemInstanceList
...
```

```
hostname, instanceNr, httpPort, httpsPort, startPriority, features, dispstatus
dc1hana1, 2, 50213, 50214, 0.3, HDB|HDB_WORKER, GREEN
dc1hana2, 2, 50213, 50214, 0.3, HDB|HDB_WORKER, GREEN
```

4. Additionally, you can verify the **landscapeHostConfiguration.py** output for status**ok**:

```
rh1adm $ cdpy; python landscapeHostConfiguration.py
| Host     | Host   | Host   | Failover | Remove | Storage   | Storage   | Failover | Failover |
NameServer | NameServer | IndexServer | IndexServer | Host   | Host   | Worker  | Worker  |
|          | Active | Status | Status   | Status | Config    | Actual    | Config   | Actual   | Config     |
Actual     | Config      | Actual      | Config | Actual | Config  | Actual  |
|          |        |        |          |        | Partition | Partition | Group    | Group    | Role       | Role       |
Role       | Role        | Roles  | Roles  | Groups  | Groups  |
| -------- | ------ | ------ | -------- | ------ | --------- | --------- | -------- | -------- | ---------- | ----------
| ----------- | ----------- | ------ | ------ | ------- | ------- |
| dc1hana1 | yes    | ok     |          |        |         1 |         1 | default  | default  | master 1   | master
| worker      | master      | worker | worker | default | default |
| dc1hana2 | yes    | ok     |          |        |         2 |         2 | default  | default  | master 2   | slave
| worker      | slave       | worker | worker | default | default |

overall host status: ok
```

5. Check that the systemd units are installed for the HANA instance and the SAP Host Agent:

```
[root]# systemctl list-unit-files --all sap* SAP*
UNIT FILE            STATE     PRESET
sapmedia.mount       generated -
saphostagent.service enabled   disabled
sapinit.service      generated -
SAPRH1_02.service    enabled   disabled
SAP.slice            static    -

5 unit files listed.
```

6. Repeat the steps on all nodes. Note that the HANA profiles contain the individual node name in the format **<SID>_HDB<instance>_<node>**.

3.7. DISABLING SAP HANA INSTANCE AUTOSTART

The cluster controls startup and shutdown of the HANA instance in a HA cluster setup. You must configure the HANA instance profile to not automatically start the instance itself.

Procedure

1. Go to the HANA instance profile directory:

```
[root]# cd /hana/shared/<SID>/profile
```

2. Edit the instance profile:

```
[root]# vi <SID>_HDB<instance>_<hostname>
```

Ensure that **Autostart** is set to**0**.

3. Repeat the previous steps for each HANA instance that will be managed as part of the HA cluster.

Verification

- Check that **Autostart = 0** is set in the instance profiles of all HANA instances that will be managed by the HA cluster:

```
[root]# grep Autostart /hana/shared/RH1/profile/*
/hana/shared/RH1/profile/RH1_HDB02_dc1hana1:Autostart = 0
/hana/shared/RH1/profile/RH1_HDB02_dc1hana2:Autostart = 0
```

Additional resources

SAP HANA Server Installation and Update Guide

3.8. ADJUSTING THE SAP HANA NAMESERVER CONFIGURATION

The HANA installer automatically configures additional nodes of one HANA site as master name server candidates, with up to 3 master name servers in total. However, when you configure the scale-out environment in a Pacemaker cluster, the HANA mechanisms to recover component issues internally can conflict the cluster reactions to health issues it detects.

Therefore, you must disable the alternative master name servers to prevent HANA from trying to recover issues with the active name server node on an alternative node at the same site. The cluster HA allows you control over the recovery by triggering an immediate takeover to the secondary HANA site.

Procedure

1. Stop the HANA instances on all nodes. Run as the **<sid>adm** user on one HANA instance on each site:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function StopSystem HDB
```

2. Verify as **<sid>adm** on all sites that the HANA instances are stopped completely and their status is **GRAY** in the instance list:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function GetSystemInstanceList
...
hostname, instanceNr, httpPort, httpsPort, startPriority, features, dispstatus
dc1hana2, 0, 50013, 50014, 0.3, HDB|HDB_WORKER, GRAY
dc1hana1, 0, 50013, 50014, 0.3, HDB|HDB_WORKER, GRAY
```

3. Change to the HANA configuration directory, as the **<sid>adm** user, using the command alias **cdcoc**, which is built into the**<sid>adm** user shell. This automatically changes into the **/hana/shared/<SID>/global/hdb/custom/config/** path:

```
rh1adm $ cdcoc
```

4. Edit the **nameserver.ini** file of the HANA site to update the**master** parameter in the **[landscape]** configuration block. Delete any additional master candidate, but keep the first master name server host entry, so that the remaining entry looks as follows:

```
…
[landscape]
id = …
master = dc1hana1:30201
worker = …
…
```

Due to the shared **/hana/shared** filesystem between the nodes of each HANA site, you only adjust the configuration once per site. Do not try to edit the same file on the shared filesystem simultaneously on more than one node of the same site.

5. Start the HANA instances on all cluster nodes. Run as **<sid>adm** on one HANA instance on each site:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function StartSystem HDB
```

Verification

1. Check the **nameserver.ini** configuration, for example, on site 1:

```
rh1adm $ cdcoc; grep -E "^\[landscape|^master" nameserver.ini
[landscape]
master = dc1hana1:30201
```

2. Verify that the topology of the running HANA landscape in both sites only shows one master in the **NameServer Config Role**, for example, in HANA site**DC1**:

```
rh1adm $ cdpy; python ./landscapeHostConfiguration.py | cut -d '|' -f 2,11-16
 Host     | NameServer | NameServer | IndexServer | IndexServer | Host   | Host
          | Config     | Actual     | Config      | Actual      | Config | Actual
          | Role       | Role       | Role        | Role        | Roles  | Roles
-------- | ---------- | ---------- | ----------- | ----------- | ------ | ------
 dc1hana1 | master 1   | master     | worker      | master      | worker | worker
 dc1hana2 | slave      | slave      | worker      | slave       | worker | worker
```

CHAPTER 4. CONFIGURING THE SAP HANA SYSTEM REPLICATION

You must configure and test the SAP HANA system replication before you can configure the HANA instance in a cluster. Follow the SAP guidelines for the HANA system replication setup: SAP HANA System Replication: Configuration.

4.1. PREREQUISITES FOR THE SAP HANA SYSTEM REPLICATION SETUP

SAP HANA configuration

SAP HANA must be installed and configured identically on the system replication sites.

Host name resolution

All hosts must be able to resolve the host names and fully qualified domain names (FQDN) of all HANA systems. To ensure that all host names can be resolved even without DNS you can place them into **/etc/hosts**. This is also recommended for hosts configured in HA clusters in general.

In addition you can manage host name resolution in SAP HANA internally. For more details see Internal Host Name Resolution and Host Name Resolution for System Replication. Ensure that the HANA internal host names and **/etc/hosts** entries are consistent.

NOTE

As documented at hostname | SAP Help Portal, SAP HANA only supports hostnames with lowercase characters.

SAP HANA log_mode

For the system replication to work, you must set the SAP HANA **log_mode** variable to **normal**, which is the default value.

Verify the current **log_mode** as the HANA administrative user **<sid>adm** on both nodes:

```
rh1adm $ hdbsql -u system -i ${TINSTANCE} \
"select value from "SYS"."M_INIFILE_CONTENTS" where key='log_mode'"
Password: <HANA_SYSTEM_PASSWORD>
VALUE "normal"
1 row selected
```

Additional resources

- SAP HANA System Replication: Configuration
- SAP Note 3221437 - System replication is failed due to "Connection refused: Primary has to run in log mode normal for system replication!"

4.2. PERFORMING AN INITIAL HANA DATABASE BACKUP

You can only enable the HANA system replication when an initial backup of the SAP HANA database exists on the primary site for the planned SAP HANA system replication setup.

You can use SAP HANA tools to create the backup and skip the manual procedure. See SAP HANA Administration Guide - SAP HANA Database Backup and Recovery for more information.

Prerequisites

- You have a writable directory to which the backup files are saved for the SAP HANA administrative user **<sid>adm**.
- You have sufficient free space available in the filesystem on which the backup files are stored.

Procedure

1. Optional: Create a dedicated directory for the backup in a suitable path, for example:

```
[root]# mkdir <path>/<SID>-backup
```

   Replace **<path>** with a path on your system, which has enough free space for the initial backup files.

2. Change the owner of the backup path to user **<sid>adm** if the target directory is not already owned or writable by the HANA user, for example:

```
[root]# chown <sid>adm:sapsys <path>/<SID>-backup
```

3. Change to the **<sid>adm** user for the remaining steps:

```
[root]# su - <sid>adm
```

4. Create a backup of the **SYSTEMDB** as the**<sid>adm** user. Specify the path to the files the backups will be stored in. Ensure that the target filesystem has enough free space left, then create the backup:

```
rh1adm $ hdbsql -i ${TINSTANCE} -u system -d SYSTEMDB \
"BACKUP DATA USING FILE ('<path>/${SAPSYSTEMNAME}-backup/bkp-SYS')"
Password: <HANA_SYSTEM_PASSWORD>
0 rows affected (overall time xx.xxx sec; server time xx.xxx sec)
```

   - **$TINSTANCE** and **$SAPSYSTEMNAME** are environment variables that are part of the **<sid>adm** user shell environment.**$TINSTANCE** is the instance number and **$SAPSYSTEMNAME** is the SID. Both are automatically set to the instance values related to the **<sid>adm** user.
   - Replace **<path>** with the path where the**<sid>adm** user has write access and where there is enough free space left.

5. Create a backup of all tenant databases as the **<sid>adm** user. Specify the path to the files the backups will be stored in. Ensure that the target filesystem has enough free space left. Create the tenant DB backup:

```
rh1adm $ hdbsql -i ${TINSTANCE} -u system -d SYSTEMDB \
```

```
"BACKUP DATA FOR ${SAPSYSTEMNAME} USING FILE ('<path>/${SAPSYSTEMNAME}-backup/bkp-${SAPSYSTEMNAME}')"
Password: <HANA_SYSTEM_PASSWORD>
0 rows affected (overall time xx.xxx sec; server time xx.xxx sec)
```

Replace **<path>** with the path where the**<sid>adm** user has write access and where there is enough free space left.

Verification

1. List the resulting backup files. Example when using **/hana/log/RH1-backup** as the directory to store the initial backup:

```
rh1adm $ ls -lh /hana/log/RH1-backup/
total 7.3G
-rw-r-----. 1 rh1adm sapsys 156K Sep 15 10:58 bkp-RH1_databackup_0_1
-rw-r-----. 1 rh1adm sapsys  81M Sep 15 10:58 bkp-RH1_databackup_2_1
-rw-r-----. 1 rh1adm sapsys 3.6G Sep 15 10:58 bkp-RH1_databackup_3_1
-rw-r-----. 1 rh1adm sapsys  81M Sep 15 10:58 bkp-RH1_databackup_4_1
-rw-r-----. 1 rh1adm sapsys 164K Sep 15 10:56 bkp-SYS_databackup_0_1
-rw-r-----. 1 rh1adm sapsys 3.6G Sep 15 10:57 bkp-SYS_databackup_1_1
```

2. Use the HANA command **hdbbackupcheck** to confirm the sanity of each backup file you created:

```
rh1adm $ for i in $(ls /hana/log/RH1-backup/*); do hdbbackupcheck $i; done
...
Loaded library 'libhdblivecache'
Backup '/hana/log/RH1-backup/RH1_databackup_0_1' successfully checked.
Loaded library 'libhdbcsaccessor'
...
Loaded library 'libhdblivecache'
Backup '/hana/log/RH1-backup/system_databackup_0_1' successfully checked.
Loaded library 'libhdbcsaccessor'
Loaded library 'libhdblivecache'
Backup '/hana/log/RH1-backup/system_databackup_1_1' successfully checked.
```

Troubleshooting

- The backup fails when the **<sid>adm** user is not able to write to the target directory:

```
* 447: backup could not be completed: [2001003]
createDirectory(path= '/tmp/RH1-backup/', access= rwxrwxr--, recursive= true):
Permission denied (rc= 13, 'Permission denied') SQLSTATE: HY000
```

  Ensure that the **<sid>adm** user can create files inside of the target directory you define in the backup command. Fix the permissions, for example using step 2 of the procedure.

- The backup fails because the target filesystem runs out of space:

```
* 447: backup could not be completed: [2110001]
Generic stream error: $msg$ - , rc=$sysrc$: $sysmsg$.
Failed to process item 0x00007fc5796e0000 - '<root>/.bkp-RH1_databackup_3_1'
((open, mode= W, file_access= rw-r-----, flags=
```

```
ASYNC|DIRECT|TRUNCATE|UNALIGNED_SIZE,
size= 4096), factory= (root= '/tmp/RH1-backup/' (root_access= rwxr-x---,
flags= AUTOCREATE_PATH|DISKFULL_ERROR, usage= DATA_BACKUP, fs= xfs,
config=
(async_write_submit_active=on,async_write_submit_blocks=all,async_read_submit=o
n,num_submit_queues=1,num_completion_queues=1,size_kernel_io_queue=512,max_
parallel_io_requests=64,min_submit_batch_size=16,max_submit_batch_size=64))
SQLSTATE: HY000
```

Check the free space of the filesystem on which the target directory is located. Increase the filesystem size or choose a different path with enough free space available for the backup files.

4.3. CONFIGURING THE PRIMARY HANA REPLICATION SITE

Enable the HANA system replication on a system which you plan to be the initial primary site of your planned system replication setup.

Prerequisites

- You have created an initial backup for the HANA database on the primary node site based on steps described in Performing an initial HANA database backup.

Procedure

1. Enable the system replication on the HANA site that becomes the initial primary. Run the command as **<sid>adm** on the first, or primary, node:

```
rh1adm $ hdbnsutil -sr_enable --name=<site1>
nameserver is active, proceeding ...
successfully enabled system as system replication source site
done.
```

- Replace **<site1>** with your primary HANA site name, for example, **DC1**.

Verification

- Check the system replication configuration as **<sid>adm**, and verify that it shows the current node as **mode: primary**, and that **site id** and **site name** are populated with the primary site information:

```
rh1adm $ hdbnsutil -sr_stateConfiguration
System Replication State
~~~~~~~~

mode: primary
site id: 1
site name: DC1
done.
```

4.4. CONFIGURING THE SECONDARY HANA REPLICATION SITE

You must register the secondary HANA site to the primary site to complete the setup of the HANA system replication.

Prerequisites

- You have installed SAP HANA on the secondary nodes using the same SID and instance number as the primary instances.
- You have configured SSH public-key access for the root user between the cluster nodes.
- You have configured the firewall rules in your network infrastructure or on each host operating system to allow the connections that your HANA landscape requires for the system replication connection.
- You have opened 2 terminals on a secondary node, for example, on **dc2hana1** with one terminal for the **root** user and one for the **<sid>adm** user.

Procedure

1. Stop the secondary HANA instances. Run as the **<sid>adm** user on one secondary instance, for example, on **dc2hana1**:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function StopSystem HDB
```

2. Run this and the following steps only on one node on the secondary site, for example, on **dc2hana1**. Change to the directory on the shared filesystem where HANA stores the keys of the system replication encryption on one node of the secondary site:

```
[root]# cd /hana/shared/<SID>/global/security/rsecssfs
```

3. Copy the HANA system PKI file **SSFS_<SID>.KEY** from the primary HANA site to the secondary site on one secondary node:

```
[root]# rsync -av <node1>:$PWD/key/SSFS_<SID>.KEY key/
```

   - Replace **<node1>** with a primary node, for example, **dc1hana1**.
   - Replace **<SID>** with your HANA SID, for example, **RH1**.

4. Copy the PKI file **SSFS_<SID>.DAT** from the primary HANA site to the secondary site on one secondary node in the same way as the previous step:

```
[root]# rsync -av <node1>:$PWD/data/SSFS_<SID>.DAT data/
```

5. Register the secondary HANA site to the primary site. Run this in the **<sid>adm** user terminal on a secondary node:

```
rh1adm $ hdbnsutil -sr_register --remoteHost=<node1> \
--remoteInstance=${TINSTANCE} --replicationMode=sync \
--operationMode=logreplay --name=<site2>
adding site ...
collecting information ...
updating local ini files ...
done.
```

–

- Replace **<node1>** with a primary node, for example, **dc1hana1**.
- Replace **<site2>** with your secondary HANA site name, for example **DC2**.
- Choose the values for **replicationMode** and **operationMode** according to your requirements for the system replication.
- **$TINSTANCE** is an environment variable that is set automatically for user **<sid>adm** by reading the HANA instance profile. The variable value is the HANA instance number.

6. Start the secondary HANA instances. Run as **<sid>adm** on one HANA instance on the secondary site:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function StartSystem HDB
```

Verification

1. Check that the system replication is running on the secondary site and the **mode** matches the value you used for the **replicationMode** parameter in the **hdbnsutil -sr_register** command. Run as **<sid>adm** on one node on the secondary site, for example **dc2hana1**:

```
rh1adm $ hdbnsutil -sr_stateConfiguration

System Replication State
~~~~~~~~

mode: sync
site id: 2
site name: DC2
active primary site: 1

primary masters: dc1hana1
```

2. Change to the Python script directory of the HANA instance as user **<sid>adm**, on both sites. The easiest way to do this is to use **cdpy**, which is an alias built into the **<sid>adm** user shell that SAP HANA populates during the instance installation:

```
rh1adm $ cdpy
```

In our example this command changes the current directory to **/usr/sap/RH1/HDB02/exe/python_support/**.

3. Show the current status of the established HANA system replication, on both HANA sites.

   a. On the primary site the system replication status is always displayed with all details:

```
rh1adm $ python systemReplicationStatus.py
|Database |Host     |Port  |Service Name |Volume ID |Site ID |Site Name |Secondary
|Secondary |Secondary |Secondary |Secondary     |Replication |Replication |Replication
|Secondary    |
|         |         |      |             |          |        |          |Host      |Port      |Site ID   |Site Name
|Active Status |Mode        |Status      |Status Details |Fully Synced |
|-------- |-------- |----- |------------ |--------- |------- |--------- |--------- |--------- |--------- |-------
```

```
-- |------------- |----------- |----------- |-------------- |------------ |
|RH1      |dc1hana2 |30003 |indexserver  |       4 |      1 |DC1       |dc2hana2  |    30003 |
2 |DC2       |YES          |SYNC        |ACTIVE      |              |        True |
|SYSTEMDB |dc1hana1 |30001 |nameserver   |       1 |      1 |DC1       |dc2hana1  |
30001 |        2 |DC2       |YES          |SYNC        |ACTIVE      |              |        True |
|RH1      |dc1hana1 |30007 |xsengine     |       2 |      1 |DC1       |dc2hana1  |    30007 |
2 |DC2       |YES          |SYNC        |ACTIVE      |              |        True |
|RH1      |dc1hana1 |30003 |indexserver  |       3 |      1 |DC1       |dc2hana1  |    30003 |
2 |DC2       |YES          |SYNC        |ACTIVE      |              |        True |

status system replication site "2": ACTIVE
overall system replication status: ACTIVE

Local System Replication State
~~~~~~~~~~

mode: PRIMARY
site id: 1
site name: DC1
```

- **ACTIVE** means that the HANA system replication is in a healthy state and fully synced.
- **SYNCING** is displayed while the system replication is being updated on the secondary site, for example after a takeover of the secondary site to become the new primary.
- **INITIALIZING** is shown after the system replication has first been enabled or a full sync has been triggered.

b. On the secondary site, the output of **systemReplicationStatus.py** is less detailed. Check the status on one secondary node:

```
rh1adm $ python systemReplicationStatus.py
this system is either not running or not primary system replication site

Local System Replication State
~~~~~~~~~~

mode: SYNC
site id: 2
site name: DC2
active primary site: 1
primary masters: dc1hana1
```

Additional resources

- SAP Note 2369981 - Required configuration steps for authentication with HANA System Replication
- Replication Modes for SAP HANA System Replication
- Operation Modes for SAP HANA System Replication

4.5. TESTING THE HANA SYSTEM REPLICATION

We recommend that you test the HANA system replication thoroughly before you proceed with the cluster setup. The verification of the correct system replication behavior can help to prevent unexpected results when the HA cluster manages the system replication afterwards.

Use timeout values in the cluster resource configuration that cover the measured times of the different tests to ensure that cluster resource operations do not time out prematurely.

You can also test different parameter values in the HANA configuration to optimize the performance by measuring the time that certain activities take when performed manually outside of cluster control.

Perform the tests with realistic data loads and sizes.

Full replication

- How long does the synchronization take after the newly registered secondary is started until it is in sync with the primary?
- Are there parameters which can improve the synchronization time?

Lost connection

- How long does it take after the connection was lost between the primary and the secondary site, until they are in sync again?
- Are there parameters which can improve the reconnection and sync times?

Takeover

- How long does the secondary site take to be fully available after a takeover from the primary?
- What is the time difference between a normal takeover and a "takeover with handshake"?
- Are there parameters which can improve the takeover time?

Data consistency

- Is the data you create available and correct after you perform a takeover?

Client reconnect

- Can the client reconnect to the new primary site after a takeover?
- How long does it take for the client to access the new primary after a takeover?

Primary becomes secondary

- How long does it take a former primary until it is in sync again with the new primary, after it is registered as a new secondary?
- If configured, how long does it take until a client can access the newly registered secondary for read operations?

Additional resources

Refer to section “9. Testing”, in How To Perform System Replication for SAP HANA, for more information.

CHAPTER 5. CONFIGURING THE PACEMAKER CLUSTER

5.1. DEPLOYING THE BASIC CLUSTER CONFIGURATION

The following basic cluster setup covers the minimum steps to get started with the Pacemaker cluster setup for managing SAP instances. Apply the steps to include all the nodes according to your planned cluster configuration.

For more information on settings and options for complex configurations, refer to the documentation for the RHEL HA Add-On, for example, Create a high availability cluster with multiple links.

Prerequisites

- You have set up the HANA system replication environment and verified that it is working correctly.
- You have configured the RHEL High Availability repository on all systems that are going to be nodes of this cluster.
- You have verified fencing and quorum requirements according to your planned environment. For more details see HA cluster requirements.

Procedure

1. Install the Red Hat High Availability Add-On software packages from the High Availability repository. Choose which fence agents you want to install and execute the installation on all cluster nodes.

```
[root]# dnf install pcs pacemaker fence-agents-<model>
```

2. Start and enable the **pcsd** service on all cluster nodes. The **--now** parameter automatically starts the enabled service:

```
[root]# systemctl enable --now pcsd.service
```

3. Optional: If you use the local **firewalld** service you must enable the ports that are required by the Red Hat High Availability Add-On. Run this on all cluster nodes:

```
[root]# firewall-cmd --add-service=high-availability
[root]# firewall-cmd --runtime-to-permanent
```

4. Set a password for the user **hacluster**. Repeat the command on each node using the same password:

```
[root]# passwd hacluster
```

5. Authenticate the user **hacluster** for each node in the cluster. Run this on one node:

```
[root]# pcs host auth <node1> <node2> <node3> <node4>
Username: hacluster
Password:
dc1hana1: Authorized
```

```
dc1hana2: Authorized
dc2hana1: Authorized
dc2hana2: Authorized
```

- Enter the node names with or without FQDN, as defined in the **/etc/hosts** file.
- Enter the **hacluster** user password in the prompt.

6. Create the cluster with a unique name and provide the names of all cluster members with fully qualified host names. This propagates the cluster configuration on all nodes and starts the cluster. Run this command on one node:

```
[root]# pcs cluster setup <cluster_name> --start <node1> <node2> <node3> <node4>
No addresses specified for host 'dc1hana1', using 'dc1hana1'
No addresses specified for host 'dc1hana2', using 'dc1hana2'
No addresses specified for host 'dc2hana1', using 'dc2hana1'
No addresses specified for host 'dc2hana2', using 'dc2hana2'
Destroying cluster on hosts: 'dc1hana1', 'dc1hana2', 'dc2hana1', 'dc2hana2'...
dc2hana2: Successfully destroyed cluster
dc1hana2: Successfully destroyed cluster
dc1hana1: Successfully destroyed cluster
dc2hana1: Successfully destroyed cluster
Requesting remove 'pcsd settings' from 'dc1hana1', 'dc1hana2', 'dc2hana1', 'dc2hana2'
dc1hana1: successful removal of the file 'pcsd settings'
dc1hana2: successful removal of the file 'pcsd settings'
dc2hana1: successful removal of the file 'pcsd settings'
dc2hana2: successful removal of the file 'pcsd settings'
Sending 'corosync authkey', 'pacemaker authkey' to 'dc1hana1', 'dc1hana2', 'dc2hana1',
'dc2hana2'
dc1hana1: successful distribution of the file 'corosync authkey'
dc1hana1: successful distribution of the file 'pacemaker authkey'
dc1hana2: successful distribution of the file 'corosync authkey'
dc1hana2: successful distribution of the file 'pacemaker authkey'
dc2hana1: successful distribution of the file 'corosync authkey'
dc2hana1: successful distribution of the file 'pacemaker authkey'
dc2hana2: successful distribution of the file 'corosync authkey'
dc2hana2: successful distribution of the file 'pacemaker authkey'
Sending 'corosync.conf' to 'dc1hana1', 'dc1hana2', 'dc2hana1', 'dc2hana2'
dc1hana1: successful distribution of the file 'corosync.conf'
dc1hana2: successful distribution of the file 'corosync.conf'
dc2hana1: successful distribution of the file 'corosync.conf'
dc2hana2: successful distribution of the file 'corosync.conf'
Cluster has been successfully set up.
Starting cluster on hosts: 'dc1hana1', 'dc1hana2', 'dc2hana1', 'dc2hana2'...
```

7. Enable the cluster to start automatically on system start on all cluster nodes, which enables the **corosync** and **pacemaker** services. Skip this step if you prefer to manually control the start of the cluster after a node restarts. Run on one node:

```
[root]# pcs cluster enable --all
dc1hana1: Cluster Enabled
dc1hana2: Cluster Enabled
dc2hana1: Cluster Enabled
dc2hana2: Cluster Enabled
```

Verification

- **Check the cluster status after the initial configuration. Verify that it shows all cluster nodes as Online and the status of all cluster daemons is active/enabled:**

```
[root]# pcs status --full
Cluster name: hana-scaleout-cluster

WARNINGS:
No stonith devices and stonith-enabled is not false

Cluster Summary:
  * Stack: corosync (Pacemaker is running)
  * Current DC: dc1hana1 (1) (version 2.1.9-1.2.el9_6-49aab9983) - partition with quorum
  * Last updated: Mon Sep  8 13:41:25 2025 on dc1hana1
  * Last change:  Mon Sep  8 13:34:45 2025 by hacluster via hacluster on dc1hana1
  * 4 nodes configured
  * 0 resource instances configured

Node List:
  * Node dc1hana1 (1): online, feature set 3.19.6
  * Node dc1hana2 (2): online, feature set 3.19.6
  * Node dc2hana1 (3): online, feature set 3.19.6
  * Node dc2hana2 (4): online, feature set 3.19.6

Full List of Resources:
  * No resources

Migration Summary:

Tickets:

PCSD Status:
  dc1hana1: Online
  dc1hana2: Online
  dc2hana1: Online
  dc2hana2: Online

Daemon Status:
  corosync: active/enabled
  pacemaker: active/enabled
  pcsd: active/enabled
```

Next steps

- **Configure a fencing method to enable the STONITH mechanism. See Configuring fencing in a Red Hat High Availability cluster.**
- **Test the fencing setup before you proceed with further configuration of the cluster. For more information, see How to test fence devices and fencing configuration in a Red Hat High Availability cluster?**
- **Configure a quorum device, see Configuring a quorum device in the cluster**

Additional resources

- Creating a Red Hat High-Availability cluster with Pacemaker
- Support Policies for RHEL High Availability Clusters
- Exploring Concepts of RHEL High Availability Clusters - Quorum
- **dnf(8)** and **systemctl(1)** man pages on your system

5.2. CONFIGURING GENERAL CLUSTER PROPERTIES

You must adjust cluster resource defaults to avoid unnecessary failovers of the resources.

Procedure

- Run the following command on one cluster node to update the default values of the **resource-stickiness** and **migration-threshold** parameters:

```
[root]# pcs resource defaults update \
resource-stickiness=1000 \
migration-threshold=5000
```

  - **resource-stickiness=1000** encourages the resource to stay running where it is. This prevents the cluster from moving the resources based on light internal health indicators.
  - **migration-threshold=5000** enables the resource to be restarted on a node after 5000 failures. After exceeding this limit, the resource is blocked on the node until the failure has been cleared. This allows resource recovery after a few failures until an administrator can investigate the cause of the repeated failure and reset the counter.

Verification

- Check that the resource defaults are set:

```
[root]# pcs resource defaults
Meta Attrs: build-resource-defaults
  migration-threshold=5000
  resource-stickiness=1000
```

5.3. CONFIGURING THE SYSTEMD-BASED SAP STARTUP FRAMEWORK

Systemd integration is the default behavior of SAP HANA installations on RHEL 9 for SAP HANA 2.0 SPS07 revision 70 and newer. In HA environments you must apply additional modifications to integrate the different systemd services that are involved in the cluster setup.

Configure the pacemaker systemd service to manage the HANA instance systemd service in the correct order on all cluster nodes running HANA instances..

Prerequisites

- You have installed the HANA instances with systemd integration and have checked the systemd integration on all HANA nodes, for example:

```
[root]# systemctl list-units --all SAP*
  UNIT              LOAD      ACTIVE   SUB     DESCRIPTION
  SAPRH1_02.service loaded    active   running SAP Instance SAPRH1_02
  SAP.slice         loaded    active   active  SAP Slice
...
```

Procedure

1. **Create the directory /etc/systemd/system/pacemaker.service.d/ for the pacemaker service drop-in file:**

```
[root]# mkdir /etc/systemd/system/pacemaker.service.d/
```

2. **Create the systemd drop-in file for the pacemaker service with the following content:**

```
[root]# cat << EOF > /etc/systemd/system/pacemaker.service.d/00-pacemaker.conf
[Unit]
Description=Pacemaker needs the SAP HANA instance service
Wants=SAP<SID>_<instance>.service
After=SAP<SID>_<instance>.service
EOF
```

   - **Replace <SID> with your HANA SID.**
   - **Replace <instance> with your HANA instance number.**

3. **Reload the systemctl daemon to activate the drop-in file:**

```
[root]# systemctl daemon-reload
```

4. **Repeat steps 1-3 on the other HANA cluster nodes.**

Verification

1. **Check the systemd service of your HANA instance and verify that it is loaded:**

```
[root]# systemctl status SAPRH1_02.service
● SAPRH1_02.service - SAP Instance SAPRH1_02
    Loaded: loaded (/etc/systemd/system/SAPRH1_02.service; disabled; preset: disabled)
    Active: active (running) since xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
  Main PID: 5825 (sapstartsrv)
     Tasks: 841
    Memory: 88.6G
       CPU: 4h 50min 2.033s
    CGroup: /SAP.slice/SAPRH1_02.service
            ├── 5825 /usr/sap/RH1/HDB02/exe/sapstartsrv
pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_dc1hana1
            ├── 5986 sapstart pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_dc1hana1
            ├── 5993 /usr/sap/RH1/HDB02/dc1hana1/trace/hdb.sapRH1_HDB02 -d -nw -f
/usr/sap/RH1/HDB02/dc1hana1/daemon.ini
pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_dc1hana1
...
```

2. Verify that the SAP HANA instance service is known to the pacemaker service now:

```
[root]# systemctl show pacemaker.service | grep -E 'Wants=|After=|SAP.{6}.service'
Wants=SAPRH1_02.service resource-agents-deps.target dbus-broker.service
After=... SAPRH1_02.service …
```

Make sure that the **SAP<SID>_<instance>.service** is listed in the **After=** and **Wants=** lists.

Additional resources

- The Systemd-Based SAP Startup Framework
- Changes required for managing systemd enabled SAP HANA or "anydb" instances managed by the RHEL HA Add-On
- SAP Note 3139184 - Linux: systemd integration for sapstartsrv and SAP Host Agent
- SAP Note 3189534 - Linux: systemd integration for sapstartsrv and SAP HANA
- **systemctl(1)** man page on your system

5.4. INSTALLING THE SAP HANA HA COMPONENTS

The **sap-hana-ha** RPM package in the **Red Hat Enterprise Linux 9 for <arch> - SAP Solutions (RPMs)** repository provides resource agents and other SAP HANA specific components for setting up a HA cluster for managing HANA system replication setup.

IMPORTANT

The package **sap-hana-ha** is only available since RHEL 9.4. If you configure a cluster for HANA HA on an older RHEL 9 release, follow the instructions in Automating SAP HANA Scale-Out System Replication using the RHEL HA Add-On instead.

Procedure

- Install the **sap-hana-ha** package on all cluster nodes:

```
[root]# dnf install sap-hana-ha
```

Verification

- Check on all nodes that the package is installed:

```
[root]# rpm -q sap-hana-ha
sap-hana-ha-<version>.<release>.noarch
```

Additional resources

- Support Policies for RHEL High Availability Clusters - Management of SAP HANA in a Cluster

5.5. CONFIGURING THE HANASR HA/DR PROVIDER FOR THE SRCONNECTIONCHANGED() HOOK METHOD

When you configure the HANA instance in a HA cluster setup with SAP HANA 2.0 SPS0 or later, you must enable and test the SAP HANA **srConnectionChanged()** hook method before proceeding with the cluster setup.

Prerequisites

- You have installed the **sap-hana-ha** package.
- Your HANA instance is not yet managed by the cluster. Otherwise, use the maintenance procedure Performing maintenance on the SAP HANA instances to make sure that the cluster does not interfere during the configuration of the hook scripts.

Procedure

1. Stop the HANA instances on all nodes. Run this as the **<sid>adm** user on one HANA instance per site:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function StopSystem HDB
```

2. Verify as **<sid>adm** on all sites that the HANA instances are stopped completely and their status is **GRAY** in the instance list. Run this on one host on each site:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function GetSystemInstanceList
...
hostname, instanceNr, httpPort, httpsPort, startPriority, features, dispstatus
dc1hana2, 0, 50013, 50014, 0.3, HDB|HDB_WORKER, GRAY
dc1hana1, 0, 50013, 50014, 0.3, HDB|HDB_WORKER, GRAY
```

3. Change to the HANA configuration directory, as the **<sid>adm** user, using the command alias **cdcoc**, which is built into the **<sid>adm** user shell. This automatically changes into the **/hana/shared/<SID>/global/hdb/custom/config/** path:

```
rh1adm $ cdcoc
```

4. Update the **global.ini** file of the SAP HANA site to configure the **HanaSR** hook. Edit the configuration file on one node of each HANA site and add the following configuration:

```
[ha_dr_provider_hanasr]
provider = HanaSR
path = /usr/share/sap-hana-ha/
execution_order = 1

[trace]
ha_dr_hanasr = info
```

Set **execution_order** to **1** to ensure that the **HanaSR** hook is always executed with the highest priority.

Due to the shared **/hana/shared** filesystem between the nodes of each HANA site, you only adjust the configuration once per site. Do not try to edit the same file on the shared filesystem simultaneously on more than one node of the same site.

5. Optional: When you also want to configure the optional **ChkSrv** hook for taking action on an **hdbindexserver** failure, you can add the changes to the **global.ini** at the same time, see step 1 in Configuring the ChkSrv HA/DR provider for the srServiceStateChanged() hook method:

```
[ha_dr_provider_hanasr]
provider = HanaSR
path = /usr/share/sap-hana-ha/
execution_order = 1

[ha_dr_provider_chksrv]
provider = ChkSrv
path = /usr/share/sap-hana-ha/
execution_order = 2
action_on_lost = stop

[trace]
ha_dr_hanasr = info
ha_dr_chksrv = info
```

6. Create the file **/etc/sudoers.d/20-saphana**, as the **root** user, on each cluster node with the following content. These command privileges allow the **<sid>adm** user to update certain cluster node attributes as part of the HanaSR hook execution:

```
[root]# visudo -f /etc/sudoers.d/20-saphana
Defaults:<sid>adm !requiretty
<sid>adm ALL=(ALL) NOPASSWD: /usr/sbin/crm_attribute -n hana_*
```

- For further information on why the **Defaults** setting is needed, refer to The srHook attribute is set to SFAIL in a Pacemaker cluster managing SAP HANA system replication, even though replication is in a healthy state.

7. Start the HANA instances on all cluster nodes manually without starting the HA cluster. Run as **<sid>adm** on one HANA instance per site:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function StartSystem HDB
```

Verification

1. Change to the SAP HANA directory, as the **<sid>adm** user, where trace log files are stored. Use the command alias **cdtrace**, which is built into the **<sid>adm** user shell:

```
rh1adm $ cdtrace
```

2. Check the HANA nameserver process logs for the HA/DR provider loading message:

   a. If only the **HanaSR** provider is configured:

```
rh1adm $ grep -he "loading HA/DR Provider.*" nameserver_*
loading HA/DR Provider 'HanaSR' from /usr/share/sap-hana-ha/
```

b. If the optional **ChkSrv** provider is also implemented:

```
rh1adm $ grep -he "loading HA/DR Provider.*" nameserver_*
loading HA/DR Provider 'ChkSrv' from /usr/share/sap-hana-ha/
loading HA/DR Provider 'HanaSR' from /usr/share/sap-hana-ha/
```

3. Verify as user **root** in the system secure log that the **sudo** command executed successfully. If the sudoers file is not correct, an error is logged when the **sudo** command is executed. Check this on the primary node on which the HANA master nameserver is running, for example, **dc1hana1**:

```
[root]# grep -e 'sudo.*crm_attribute.*' /var/log/secure
 sudo[4267]:  rh1adm : PWD=/hana/shared/RH1/HDB02/dc1hana1 ; USER=root ;
COMMAND=/usr/sbin/crm_attribute -n hana_rh1_site_srHook_DC2 -v SFAIL -t crm_config -s
SAPHanaSR
 ...
 sudo[4319]:  rh1adm : PWD=/hana/shared/RH1/HDB02/dc1hana1 ; USER=root ;
COMMAND=/usr/sbin/crm_attribute -n hana_rh1_site_srHook_DC2 -v SOK -t crm_config -s
SAPHanaSR
```

After the HANA instance starts on both nodes, you can usually see several **srHook** attribute updates. At first it is setting **SFAIL**, because immediately after the primary starts, it is not yet in sync with the secondary, which is still synchronizing at this time.

The last update to **SOK** is triggered by the HANA event after the system replication status finally changes to fully in sync.

4. Repeat the verification steps 1-2 on the second site, if not already done at the same time. The **sudo** log messages of step 3 are only visible on the primary instance master nameserver node, on which the system replication events are logged.

5. Check the cluster attributes on any node and verify that the value for the **hana_<sid>_site_srHook_<site2>** attribute is updated as expected:

```
[root]# cibadmin --query | grep -e 'SAPHanaSR.*srHook'
        <nvpair id="SAPHanaSR-hana_rh1_site_srHook_DC2"
name="hana_rh1_site_srHook_DC2" value="SOK"/>
```

- **SOK** is set when the HANA system replication is in **ACTIVE** state, which means established and fully in sync.
- **SFAIL** is set when the system replication is in any other state.

Troubleshooting

- Refer to The HANA instance does not start after hook changes.

Additional resources

- HA/DR provider options for HanaSR
- Install and Configure a HA/DR Provider Script

- How can the srConnectionChanged() hook be used to improve the detection of situations where a takeover is required, in a Red Hat Pacemaker cluster managing HANA Scale-up or Scale-out System Replication?
- **sudo(8)** and **visudo(8)** man pages on your system

5.6. CONFIGURING THE CHKSRV HA/DR PROVIDER FOR THE SRSERVICESTATECHANGED() HOOK METHOD

You can configure the hook **ChkSrv** if you want the HANA instance to be stopped or killed for faster recovery after an indexserver process has failed. This configuration is optional.

Prerequisites

- You have installed the **sap-hana-ha** package.
- You have configured the **HanaSR** HA/DR provider. For more information see Configuring the HanaSR HA/DR provider for the srConnectionChanged() hook method.

Procedure

1. Change to the HANA configuration directory as the **<sid>adm** user. Use the command alias **cdcoc**, which is built into the **<sid>adm** user shell. This automatically changes into the **/hana/shared/<SID>/global/hdb/custom/config/** path:

```
rh1adm $ cdcoc
```

2. Update the **global.ini** file of the HANA site to configure the hook script. Edit the configuration file one node of each HANA site and add the following content in addition to the **HanaSR** provider definition:

```
[ha_dr_provider_chksrv]
provider = ChkSrv
path = /usr/share/sap-hana-ha/
execution_order = 2
action_on_lost = stop

[trace]
ha_dr_hanasr = info
ha_dr_chksrv = info
```

   Due to the shared **/hana/shared** filesystem between the nodes of each HANA site, you only adjust the configuration once per site. Do not try to edit the same file on the shared filesystem simultaneously on more than one node of the same site.

3. Optional: If you want to use the **fence** option for **action_on_lost**, you must add the **SAPHanaSR-hookHelper** to the sudo configuration of the **<sid>adm** user:

```
[root]# visudo -f /etc/sudoers.d/20-saphana
…
<sid>adm ALL=(ALL) NOPASSWD: /usr/bin/SAPHanaSR-hookHelper
```

4. Optional: Activate the **ChkSrv** provider while HANA is running by reloading the HA/DR providers. Skip this step when configuring the hook script while the instance is down, the HA/DR provider is loaded automatically at the next instance start.

```
rh1adm $ hdbnsutil -reloadHADRProviders
```

Verification

1. Change to the SAP HANA directory, as the **<sid>adm** user, where trace log files are stored. Use the command alias **cdtrace**, which is built into the**<sid>adm** user shell:

```
rh1adm $ cdtrace
```

2. Check that the changes are loaded:

```
rh1adm $ grep -e "loading HA/DR Provider.*ChkSrv.*" nameserver_*
loading HA/DR Provider 'ChkSrv' from /usr/share/sap-hana-ha/
```

3. Check that the dedicated trace log file is created and the provider loaded with the correct configuration parameters:

```
rh1adm $ cat nameserver_chksrv.trc
init called
ChkSrv.init() version 1.001.1, parameter info: action_on_lost=stop stop_timeout=20
kill_signal=9
...
```

Troubleshooting

- Refer to The HANA instance does not start after hook changes

Additional resources

- HA/DR provider options for ChkSrv
- Implementing a HA/DR Provider

5.7. CREATING THE HANA CLUSTER RESOURCES

You must configure the **SAPHanaTopology** and **SAPHanaController** resources so that the cluster can collect the status of the HANA landscape, monitor the instance health and take action to manage the instance when required.

The **SAPHanaFilesystem** resource is optional, you can add it for improving the time to action in case the filesystem of the primary instance becomes unavailable.

Prerequisites

- You have installed the cluster and you have configured all HANA nodes in the cluster.
- You have configured the HANA system replication between your HANA sites.
- All HANA instances are running and the system replication is healthy.

Procedure

1. Create the **SAPHanaTopology** resource as a clone resource, which means it runs on all cluster nodes at the same time:

```
[root]# pcs resource create rsc_SAPHanaTop_<SID>_HDB<instance> \
ocf:heartbeat:SAPHanaTopology \
SID=<SID> \
InstanceNumber=<instance> \
op start timeout=600 \
op stop timeout=300 \
op monitor interval=30 timeout=300 \
clone cln_SAPHanaTop_<SID>_HDB<instance> \
meta clone-node-max=1 interleave=true --future
```

- Replace **<SID>** with your HANA SID.
- Replace **<instance>** with your HANA instance number.

> **NOTE**
>
> Since RHEL 9.3, a deprecation warning is displayed when not using the **meta** keyword in the clone command and the attributes are automatically assigned to the base resource.
>
> In the future, the **meta** keyword for clone attributes will be required for assigning attributes to the clone resource. Until then, add the **--future** parameter to already apply this behavior now.
>
> See also New pcs parsing requires meta keyword when specifying clone meta attributes.

2. Create the **SAPHanaController** resource as a promotable clone resource. This means it runs on all cluster nodes at the same time, but on one node it functions as the active, or primary, resource:

```
[root]# pcs resource create rsc_SAPHanaCon_<SID>_HDB<instance> \
ocf:heartbeat:SAPHanaController \
SID=<SID> \
InstanceNumber=<instance> \
PREFER_SITE_TAKEOVER=true \
DUPLICATE_PRIMARY_TIMEOUT=7200 \
AUTOMATED_REGISTER=false \
op stop timeout=3600 \
op monitor interval=59 role=Promoted timeout=700 \
op monitor interval=61 role=Unpromoted timeout=700 \
meta priority=100 \
promotable cln_SAPHanaCon_<SID>_HDB<instance> \
meta clone-node-max=1 interleave=true --future
```

We recommend that you create the resource with **AUTOMATED_REGISTER=false** and then verify the correct behavior and data consistency through tests to complete the setup. For more information, see Testing the setup. You can enable this already at creation time by setting the parameter to true.

See SAPHanaController resource parameters for more details.

3. Optional: If you want the cluster to fence the node when the **SAPHanaController** resource fails, then update the resource with the **ON_FAIL_ACTION** parameter and set it to **fence**:

```
[root]# pcs resource update rsc_SAPHanaCon_<SID>_HDB<instance>
ON_FAIL_ACTION=fence
```

In the case that you want to enable the fencing action on failure of the **SAPHanaController** resource, you must also configure a sudo entry for the helper script, which the resource agent calls to trigger the fencing:

```
[root]# visudo -f /etc/sudoers.d/20-saphana
…
<sid>adm ALL=(ALL) NOPASSWD: /usr/bin/SAPHanaSR-hookHelper
```

4. You must start the **SAPHanaTopology** resource before the **SAPHanaController** resource, because it collects HANA landscape information, which the **SAPHanaController** resource requires to start correctly. Create the cluster constraint that enforces the correct start order of the two resources:

```
[root]# pcs constraint order cln_SAPHanaTop_<SID>_HDB<instance> \
then cln_SAPHanaCon_<SID>_HDB<instance> symmetrical=false
```

Setting **symmetrical=false** indicates that the constraint only influences the start order of the resources, but it does not apply to the stop order.

5. Optional: Create the **SAPHanaFilesystem** resource as a clone resource:

```
[root]# pcs resource create rsc_SAPHanaFil_<SID>_HDB<instance> \
ocf:heartbeat:SAPHanaFilesystem \
SID=<SID> \
InstanceNumber=<instance> \
ON_FAIL_ACTION="fence" \
op start interval=0 timeout=10 \
op stop interval=0 timeout=20 \
op monitor interval=120 timeout=120 \
clone cln_SAPHanaFil_<SID>_HDB<instance> \
meta clone-node-max=1 interleave=true --future
```

The **SAPHanaFilesystem** resource agent monitors the filesystem for the **/hana/shared/<SID>/** path. If you want to change the path or configure additional **SAPHanaFilesystem** resources to monitor other paths you can use the **DIRECTORY** parameter to define the path.

Instead of setting **ON_FAIL_ACTION=fence** you can set it to **ignore**. This can be useful for testing the functionality first. The resource writes information to the system logs, which you can examine for evaluating if the resource takes the wanted action when activated to use the **fence** action.

Verification

1. Review the **SAPHanaTopology** resource clone. For operations that you do not define at resource creation, **pcs** automatically apply default values. Example resource configuration:

```
[root]# pcs resource config cln_SAPHanaTop_RH1_HDB02
Clone: cln_SAPHanaTop_RH1_HDB02
  Meta Attributes: cln_SAPHanaTop_RH1_HDB02-meta_attributes
    clone-node-max=1
    interleave=true
  Resource: rsc_SAPHanaTop_RH1_HDB02 (class=ocf provider=heartbeat
type=SAPHanaTopology)
    Attributes: rsc_SAPHanaTop_RH1_HDB02-instance_attributes
      InstanceNumber=02
      SID=RH1
    Operations:
      methods: rsc_SAPHanaTop_RH1_HDB02-methods-interval-0s
        interval=0s timeout=5
      monitor: rsc_SAPHanaTop_RH1_HDB02-monitor-interval-30
        interval=30 timeout=300
      reload: rsc_SAPHanaTop_RH1_HDB02-reload-interval-0s
        interval=0s timeout=5
      start: rsc_SAPHanaTop_RH1_HDB02-start-interval-0s
        interval=0s timeout=600
      stop: rsc_SAPHanaTop_RH1_HDB02-stop-interval-0s
        interval=0s timeout=300
```

2. **Review the SAPHanaController resource clone. Example resource configuration:**

```
[root]# pcs resource config cln_SAPHanaCon_RH1_HDB02
Clone: cln_SAPHanaCon_RH1_HDB02
  Meta Attributes: cln_SAPHanaCon_RH1_HDB02-meta_attributes
    clone-node-max=1
    interleave=true
    promotable=true
  Resource: rsc_SAPHanaCon_RH1_HDB02 (class=ocf provider=heartbeat
type=SAPHanaController)
    Attributes: rsc_SAPHanaCon_RH1_HDB02-instance_attributes
      AUTOMATED_REGISTER=false
      DUPLICATE_PRIMARY_TIMEOUT=7200
      InstanceNumber=02
      PREFER_SITE_TAKEOVER=true
      SID=RH1
    Meta Attributes: rsc_SAPHanaCon_RH1_HDB02-meta_attributes
      priority=100
    Operations:
      demote: rsc_SAPHanaCon_RH1_HDB02-demote-interval-0s
        interval=0s timeout=320
      methods: rsc_SAPHanaCon_RH1_HDB02-methods-interval-0s
        interval=0s timeout=5
      monitor: rsc_SAPHanaCon_RH1_HDB02-monitor-interval-59
        interval=59 timeout=700 role=Promoted
      monitor: rsc_SAPHanaCon_RH1_HDB02-monitor-interval-61
        interval=61 timeout=700 role=Unpromoted
      promote: rsc_SAPHanaCon_RH1_HDB02-promote-interval-0s
        interval=0s timeout=900
      reload: rsc_SAPHanaCon_RH1_HDB02-reload-interval-0s
        interval=0s timeout=5
      start: rsc_SAPHanaCon_RH1_HDB02-start-interval-0s
```

```
        interval=0s timeout=3600
      stop: rsc_SAPHanaCon_RH1_HDB02-stop-interval-0s
        interval=0s timeout=3600
```

3. **Check that the start order constraint is in place:**

```
[root]# pcs constraint order
Order Constraints:
  start resource 'cln_SAPHanaTop_RH1_HDB02' then start resource
'cln_SAPHanaCon_RH1_HDB02'
    symmetrical=0
```

4. **Optional: Review the SAPHanaFilesystem resource clone if you created it. Example resource configuration:**

```
[root]# pcs resource config cln_SAPHanaFil_RH1_HDB02
Clone: cln_SAPHanaFil_RH1_HDB02
  Meta Attributes: cln_SAPHanaFil_RH1_HDB02-meta_attributes
    clone-node-max=1
    interleave=true
  Resource: rsc_SAPHanaFil_RH1_HDB02 (class=ocf provider=heartbeat
type=SAPHanaFilesystem)
    Attributes: rsc_SAPHanaFil_RH1_HDB02-instance_attributes
      InstanceNumber=02
      ON_FAIL_ACTION=fence
      SID=RH1
    Operations:
      methods: rsc_SAPHanaFil_RH1_HDB02-methods-interval-0s
        interval=0s timeout=5
      monitor: rsc_SAPHanaFil_RH1_HDB02-monitor-interval-120
        interval=120 timeout=120
      reload: rsc_SAPHanaFil_RH1_HDB02-reload-interval-0s
        interval=0s timeout=5
      start: rsc_SAPHanaFil_RH1_HDB02-start-interval-0
        interval=0 timeout=10
      stop: rsc_SAPHanaFil_RH1_HDB02-stop-interval-0
        interval=0 timeout=20
```

5. **Check the cluster status. Use --full to include node attributes, which are updated by the HANA resources:**

```
[root]# pcs status --full
...

Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]:
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc1hana2
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc2hana1
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc2hana2
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc1hana1
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02]
```

```
(promotable):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc1hana2
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc2hana1
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc2hana2
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Promoted
dc1hana1
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02]:
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc1hana2
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc2hana1
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc2hana2
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc1hana1

Node Attributes:
  * Node: dc1hana1 (1):
    * hana_rh1_clone_state              : PROMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC1
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc1hana1
    * master-rsc_SAPHanaCon_RH1_HDB02   : 150
  * Node: dc1hana2 (2):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : slave:slave:worker:slave
    * hana_rh1_site                     : DC1
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc1hana2
    * master-rsc_SAPHanaCon_RH1_HDB02   : -10000
  * Node: dc2hana1 (3):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC2
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc2hana1
    * master-rsc_SAPHanaCon_RH1_HDB02   : 100
  * Node: dc2hana2 (4):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : slave:slave:worker:slave
    * hana_rh1_site                     : DC2
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc2hana2
    * master-rsc_SAPHanaCon_RH1_HDB02   : -12200

...
```

NOTE

The timeouts shown for the resource operations are only recommended defaults and can be adjusted depending on your SAP HANA environment. For example, large SAP HANA databases can take longer to start up and therefore you might have to increase the start timeout.

WARNING

Setting **AUTOMATED_REGISTER** to **true** can potentially increase the risk of data loss or corruption. If the HA cluster triggers a takeover when the data on the secondary HANA instance is not fully in sync, the automatic registration of the old primary HANA instance as the new secondary HANA instance results a data loss on this instance and any data that was not synced before the takeover occurred is lost as well.

For more information, see the article on the SAP Technology Blog for Members: Be Prepared for Using Pacemaker Cluster for SAP HANA – Part 2: Failure of Both Nodes.

Additional resources

- Component options
- You can also view all available resource options using **pcs resource describe <resource_agent_name>**.
- **pcs(8)**, **ocf_heartbeat_SAPHanaTopology(7)**, **ocf_heartbeat_SAPHanaController(7)** and **ocf_heartbeat_SAPHanaFilesystem(7)** man pages on your system

5.8. CREATING THE VIRTUAL IP RESOURCE

You must configure a virtual IP (VIP) resource for SAP clients to access the primary HANA instance independently from the cluster node it is currently running on. Configure the VIP resource to automatically move to the node where the primary instance is running.

The resource agent you need for the VIP resource depends on the platform you use. We are using the **IPaddr2** resource agent to demonstrate the setup.

Prerequisites

- You have reserved a virtual IP for the service.

Procedure

1. Use the appropriate resource agent for managing the virtual IP address based on the platform on which the HA cluster is running. Adjust the parameters according to the resource agent you are using. Create the cluster resource for the primary virtual IP, for example, using the **IPaddr2** agent:

```
[root]# pcs resource create rsc_vip_<SID>_HDB<instance>_primary \
ocf:heartbeat:IPaddr2 ip=<address> cidr_netmask=<netmask> nic=<device>
```

- Replace **<SID>** with your HANA SID.
- Replace **<instance>** with your HANA instance number.
- Replace **<address>**, **<netmask>** and **<device>** with the details of your primary virtual IP address.

2. Create a cluster constraint that places the VIP resource with the **SAPHanaController** resource on the HANA primary node:

```
[root]# pcs constraint colocation add rsc_vip_<SID>_HDB<instance>_primary \
with promoted cln_SAPHanaCon_<SID>_HDB<instance> 2000
```

The constraint applies a score of **2000** instead of the default **INFINITY**. This softens the resource dependency and allows the virtual IP resource to stay active in the case when there is no promoted **SAPHanaController** resource. This way it is still possible to use tools like the SAP Management Console (MMC) or SAP Landscape Management (LaMa) that can reach this IP address to query the status information of the HANA instance.

Verification

1. Check the resource configuration of the virtual IP resource, for example::

```
[root]# pcs resource config rsc_vip_RH1_HDB02_primary
Resource: rsc_vip_RH1_HDB02_primary (class=ocf provider=heartbeat type=IPaddr2)
  Attributes: rsc_vip_RH1_HDB02_primary-instance_attributes
    cidr_netmask=32
    ip=192.168.1.100
    nic=eth0
  Operations:
    monitor: rsc_vip_RH1_HDB02_primary-monitor-interval-10s
      interval=10s timeout=20s
    start: rsc_vip_RH1_HDB02_primary-start-interval-0s
      interval=0s timeout=20s
    stop: rsc_vip_RH1_HDB02_primary-stop-interval-0s
      interval=0s timeout=20s
```

2. Check that the constraint is defined correctly:

```
[root]# pcs constraint colocation
Colocation Constraints:
  Started resource 'rsc_vip_RH1_HDB02_primary' with Promoted resource
'cln_SAPHanaCon_RH1_HDB02'
    score=2000
```

3. Check that the resource is running on the promoted primary node, for example, **dc1hana1**:

```
[root]# pcs status resources rsc_vip_RH1_HDB02_primary
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):        Started dc1hana1
```

Additional resources

Additional resources

- **pcs(8)** man page on your system

5.9. ADDING A SECONDARY (READ-ENABLED) VIRTUAL IP ADDRESS

To support the Active/Active (read-enabled) secondary setup you must add a second virtual IP to provide client access to the secondary SAP HANA site.

Configure additional rules to ensure that the second virtual IP is always associated with a healthy SAP HANA site, maximizing client access and availability.

- Normal operation
  When both primary and secondary SAP HANA sites are active and the system replication is in sync, the second virtual IP is assigned to the secondary instance.

- Secondary unavailable or out of sync
  If the secondary site is down or the system replication is not in sync, the virtual IP moves to the primary instance. It automatically returns to the secondary instance as soon as the system replication is back in sync.

- Failover scenario
  In case the cluster triggers a takeover, the virtual IP remains on the same instance. After the former primary instance takes over the secondary role and the system replication is in sync again, this VIP shifts there accordingly.

Prerequisites

- You have set **operationMode=logreplay_readaccess** when registering the secondary SAP HANA instance for system replication with the primary site.

Procedure

1. Use the appropriate resource agent for managing the virtual IP address based on the platform on which the HA cluster is running. Adjust the parameters according to the resource agent you are using. Create the cluster resource for the secondary virtual IP, for example, using the **IPaddr2** agent:

   ```
   [root]# pcs resource create rsc_vip_<SID>_HDB<site>_readonly \
   ocf:heartbeat:IPaddr2 ip=<address> cidr_netmask=<netmask> nic=<device>
   ```

   - Replace **<SID>** with your HANA SID.
   - Replace **<site>** with your HANA site number.
   - Replace **<address>**, **<netmask>** and **<device>** with the details of your read-only secondary virtual IP address.

2. Create a location constraint rule to ensure that the secondary virtual IP is assigned to the secondary site during normal operations:

   ```
   [root]# pcs constraint location rsc_vip_<SID>_HDB<site>_readonly \
   rule score=INFINITY master-rsc_SAPHanaCon_<SID>_HDB<site> eq 100 \
   and hana_<sid>_clone_state eq DEMOTED
   ```

- Replace **<SID>** with your HANA SID.
- Replace **<sid>** with the lower-case HANA SID.
- Replace **<site>** with your HANA site number.

3. **Create a location constraint rule to ensure that the secondary virtual IP runs on the primary site as an alternative whenever necessary:**

```
[root]# pcs constraint location rsc_vip_<SID>_HDB<site>_readonly \
rule score=2000 master-rsc_SAPHanaCon_<SID>_HDB<site> eq 150 \
and hana_<sid>_clone_state eq PROMOTED
```

Verification

1. **Check the resource configuration of the secondary virtual IP resource, for example:**

```
[root]# pcs resource config rsc_vip_RH1_HDB02_readonly
Resource: rsc_vip_RH1_HDB02_readonly (class=ocf provider=heartbeat type=IPaddr2)
  Attributes: rsc_vip_RH1_HDB02_readonly-instance_attributes
    cidr_netmask=32
    ip=192.168.1.200
    nic=eth0
  Operations:
    monitor: rsc_vip_RH1_HDB02_readonly-monitor-interval-10s
      interval=10s timeout=20s
    start: rsc_vip_RH1_HDB02_readonly-start-interval-0s
      interval=0s timeout=20s
    stop: rsc_vip_RH1_HDB02_readonly-stop-interval-0s
      interval=0s timeout=20s
```

2. **Check that the constraints are part of the cluster configuration:**

```
[root]#  pcs constraint location
Location Constraints:
  resource 'rsc_vip_RH1_HDB02_readonly'
    Rules:
      Rule: boolean-op=and score=INFINITY
        Expression: master-rsc_SAPHanaCon_RH1_HDB02 eq 100
        Expression: hana_rh1_clone_state eq DEMOTED
  resource 'rsc_vip_RH1_HDB02_readonly'
    Rules:
      Rule: boolean-op=and score=2000
        Expression: master-rsc_SAPHanaCon_RH1_HDB02 eq 150
        Expression: hana_rh1_clone_state eq PROMOTED
```

3. **Check that the resource is running on the main secondary node, for example, dc2hana1:**

```
[root]# pcs status resources rsc_vip_RH1_HDB02_readonly
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):        Started dc2hana1
```

Additional resources

- SAP HANA System Replication - Active/Active (Read Enabled)

- **pcs(8)** man page on your system

5.10. CONFIGURING ALERT-FENCING OF THE HANA SITE

In a scale-out HANA environment it can take some time until the remaining nodes of a HANA site are able to fully stop their instances as part of the recovery after the cluster fenced a node of this site.

To reduce the time for all nodes of one HANA site to stop and the cluster to initiate a takeover to the secondary side, you can configure an alert agent that fences the remaining nodes of the same site on which the node was fenced.

The alert is called by Pacemaker after it successfully completes the initial fencing action.

Procedure

1. Create the alert in the Pacemaker cluster:

```
[root]# pcs alert create id=alert-hana-1 \
path=/usr/bin/SAPHanaSR-alert-fencing \
options alert_uptime_threshold=300
```

   - **id** is the name of the alert, for example, **alert-hana-1**.
   - **alert_uptime_threshold** is an alert option that defines the uptime in seconds after a system reboot, during which the cluster does not trigger the alert. This prevents potential fencing loops. The default is **300** seconds.

2. Add a filter to the alert configuration, which limits the cluster to only check cluster node fencing events for alert trigger activity. This prevents processing for unrelated events and reduces logging overhead. Use the **cibadmin** command to add the alert filter in XML format:

```
[root]# cibadmin --modify --scope alerts \
--xml-text '
<alert id="alert-hana-1">
 <select>
  <select_nodes />
  <select_fencing />
 </select>
</alert>'
```

   - **alert id** must be the alert name that you used in the previous step, for example, **alert-hana-1**.
   - Ensure that you surround the xml tags with single quotes **'...'**.

Verification

1. Check that the alert is defined:

```
[root]# pcs alert
Alerts:
 Alert: alert-hana-1 (path=/usr/bin/SAPHanaSR-alert-fencing)
  Options: alert_uptime_threshold=300
```

2. **Check that the alert filters are set in the cluster configuration and the XML output looks similar to this example between the <select> tags:**

```
[root]# pcs cluster cib scope=alerts
<alerts>
  <alert id="alert-hana-1" path="/usr/bin/SAPHanaSR-alert-fencing">
    <instance_attributes id="alert-hana-1-instance_attributes">
      <nvpair id="alert-hana-1-instance_attributes-alert_uptime_threshold"
name="alert_uptime_threshold" value="300"/>
    </instance_attributes>
    <select>
      <select_nodes/>
      <select_fencing/>
    </select>
  </alert>
</alerts>
```

Additional resources

- Triggering scripts for cluster events
- **cibadmin(8)** man page on your system

CHAPTER 6. CONFIGURING A QUORUM DEVICE IN THE CLUSTER

We recommend that you configure a qdevice in your cluster for improved service resiliency. Alternatively you can configure a dedicated cluster node that only serves for adding a quorum vote.

> **WARNING**
>
> Do not configure both a qdevice and a majority-maker node in the same cluster. This adds one vote per method and results in an even number of quorum votes again.

6.1. CONFIGURING A QDEVICE FOR CLUSTER QUORUM

If you prefer to use a dedicated majority-maker cluster node for this purpose, skip the qdevice setup and follow the steps in Configuring a majority-maker node for cluster quorum instead.

6.1.1. Preparing the quorum device host

- Configure the RHEL High Availability repository on the quorum device host.
- If the **firewalld** service is installed and you are not using it on your hosts, disable the service. See Disabling the firewalld service.

6.1.2. Configuring a qdevice on a quorum device host

At first you must configure a quorum device host that serves the qdevice for your cluster quorum.

In the following steps the example name of the qdevice host is **dc3qdevice**.

Prerequisites

- You have installed a separate host that is ideally located in a different location or availability zone than your cluster nodes.
- You have configured the RHEL High Availability repository on the dedicated quorum device host.
- You have configured your network in a way that your cluster nodes can reach the quorum device host.

Procedure

1. Install **pcs** and **corosync-qnetd** on the quorum device host:

```
[root]# dnf install pcs corosync-qnetd
```

2. Start and enable the **pcsd** service on the quorum device host:

```
[root]# systemctl enable --now pcsd.service
```

3. Create the qdevice on the quorum device host. This command configures and starts the quorum device model **net** and configures the device to start on boot. Run this command on the quorum device host:

```
[root]# pcs qdevice setup model net --enable --start
Quorum device 'net' initialized
quorum device enabled
Starting quorum device...
quorum device started
```

4. Optional: If you are running the **firewalld** service, enable the ports that are required by the Red Hat High Availability Add-On. Run this on the quorum device host:

```
[root]# firewall-cmd --add-service=high-availability
[root]# firewall-cmd --runtime-to-permanent
```

5. Set a password for the user **hacluster** on the quorum device host:

```
[root]# passwd hacluster
```

Verification

- Check the quorum device status on the quorum device host:

```
[root]# pcs qdevice status net --full
QNetd address:                  *:5403
TLS:                            Supported (client certificate required)
Connected clients:              0
Connected clusters:             0
Maximum send/receive size:      32768/32768 bytes
```

Additional resources

- Configuring quorum devices
- **pcs(8)** and **corosync-qdevice(8)** man pages on your system

6.1.3. Configuring a qdevice in the cluster

Prerequisites

- You have configured a quorum device host that is ideally located in a different location or availability zone than your cluster nodes, for example, **dc3qdevice**.
- You have configured a qdevice on the quorum device host.
- You have configured your network in a way that your cluster nodes can reach the quorum device host.

Procedure

1. **Add the qdevice host to the /etc/hosts on all existing cluster nodes, so that the resulting /etc/hosts entries are the same on all nodes. This ensures that the cluster nodes can communicate with the host even when the DNS services failed:**

```
[root]# cat /etc/hosts
...
192.168.100.101 dc1hana1.example.com dc1hana1
192.168.100.102 dc1hana2.example.com dc1hana2
192.168.100.103 dc2hana1.example.com dc2hana1
192.168.100.104 dc2hana2.example.com dc2hana2
192.168.100.120 dc3qdevice.example.com dc3qdevice
```

2. **Install corosync-qdevice on all nodes of your cluster:**

```
[root]# dnf install corosync-qdevice
```

3. **Authenticate the quorum device host in the cluster to enable communication. Run this command on one cluster node:**

```
[root]# pcs host auth <qdevice_host>
Username: hacluster
Password:
dc3qdevice: Authorized
```

   - **Replace <qdevice_host> with the name of your quorum device host, for example, dc3qdevice.**

4. **Add the qdevice from the quorum device host to the cluster. Run this command on one cluster node:**

```
[root]# pcs quorum device add model net host=<qdevice_host> algorithm=ffsplit
Setting up qdevice certificates on nodes...
dc2hana1: Succeeded
dc1hana2: Succeeded
dc2hana2: Succeeded
dc1hana1: Succeeded
Enabling corosync-qdevice...
dc2hana1: corosync-qdevice enabled
dc1hana1: corosync-qdevice enabled
dc1hana2: corosync-qdevice enabled
dc2hana2: corosync-qdevice enabled
Sending updated corosync.conf to nodes...
dc2hana1: Succeeded
dc1hana1: Succeeded
dc1hana2: Succeeded
dc2hana2: Succeeded
dc1hana1: Corosync configuration reloaded
Starting corosync-qdevice...
dc2hana1: corosync-qdevice started
dc1hana2: corosync-qdevice started
dc1hana1: corosync-qdevice started
dc2hana2: corosync-qdevice started
```

   - **Replace <qdevice_host> with the name of your quorum device host, for example**

- Replace **<qdevice_host>** with the name of your quorum device host, for example, **dc3qdevice**.
- The algorithm can be **ffsplit** or **lms**. Consult the **corosync-qdevice(8)** man page for more details about the different algorithms.

Verification

1. **Check the quorum configuration on a cluster node:**

```
[root]# pcs quorum config
Device:
  votes: 1
  Model: net
    algorithm: ffsplit
    host: dc3qdevice
```

2. **Check the quorum status on a cluster node:**

```
[root]# pcs quorum status
Quorum information
------------------
Date:             Wed Sep  3 14:01:24 2025
Quorum provider:  corosync_votequorum
Nodes:            4
Node ID:          1
Ring ID:          1.180
Quorate:          Yes

Votequorum information
----------------------
Expected votes:   5
Highest expected: 5
Total votes:      5
Quorum:           3
Flags:            Quorate Qdevice

Membership information
----------------------
    Nodeid      Votes    Qdevice Name
         1          1    A,V,NMW dc1hana1 (local)
         2          1    A,V,NMW dc1hana2
         3          1    A,V,NMW dc2hana1
         4          1    A,V,NMW dc2hana2
         0          1            Qdevice
```

3. **Check the quorum device status on a cluster node:**

```
[root]# pcs quorum device status
Qdevice information
-------------------
Model:                  Net
Node ID:                1
Configured node list:
    0   Node ID = 1
```

```
    1   Node ID = 2
    2   Node ID = 3
    3   Node ID = 4
Membership node list:   1, 2, 3, 4

Qdevice-net information
----------------------
Cluster name:           hana-scaleout-cluster
QNetd host:             dc3qdevice:5403
Algorithm:              Fifty-Fifty split
Tie-breaker:            Node with lowest node ID
State:                  Connected
```

4. **Check the quorum device status on the quorum device host. The status now shows the details of the cluster on which the qdevice is used. If the same qdevice is configured in multiple clusters, the status contains the details for each cluster:**

```
[root]# pcs qdevice status net --full
QNetd address:                  *:5403
TLS:                            Supported (client certificate required)
Connected clients:              4
Connected clusters:             1
Maximum send/receive size:      32768/32768 bytes
Cluster "hana-scaleout-cluster":
    Algorithm:          Fifty-Fifty split (KAP Tie-breaker)
    Tie-breaker:        Node with lowest node ID
    Node ID 1:
        Client address:         ::ffff:10.99.30.121:53312
        HB interval:            8000ms
        Configured node list:   1, 2, 3, 4
        Ring ID:                1.180
        Membership node list:   1, 2, 3, 4
        Heuristics:             Undefined (membership: Undefined, regular: Undefined)
        TLS active:             Yes (client certificate verified)
        Vote:                   ACK (ACK)
    Node ID 2:
        Client address:         ::ffff:10.99.30.57:63610
        HB interval:            8000ms
        Configured node list:   1, 2, 3, 4
        Ring ID:                1.180
        Membership node list:   1, 2, 3, 4
        Heuristics:             Undefined (membership: Undefined, regular: Undefined)
        TLS active:             Yes (client certificate verified)
        Vote:                   ACK (ACK)
    Node ID 3:
        Client address:         ::ffff:10.99.30.120:26758
        HB interval:            8000ms
        Configured node list:   1, 2, 3, 4
        Ring ID:                1.180
        Membership node list:   1, 2, 3, 4
        Heuristics:             Undefined (membership: Undefined, regular: Undefined)
        TLS active:             Yes (client certificate verified)
        Vote:                   ACK (ACK)
    Node ID 4:
        Client address:         ::ffff:10.99.30.158:54932
        HB interval:            8000ms
```

```
Configured node list:   1, 2, 3, 4
Ring ID:                1.180
Membership node list:   1, 2, 3, 4
Heuristics:             Undefined (membership: Undefined, regular: Undefined)
TLS active:             Yes (client certificate verified)
Vote:                   No change (ACK)
```

Additional resources

- Configuring quorum devices
- **pcs(8)** and **corosync-qdevice(8)** man pages on your system

6.2. CONFIGURING A MAJORITY-MAKER NODE FOR CLUSTER QUORUM

You can use an additional cluster node to serve as an extra quorum vote. In the following steps we configure such a majority-maker node **dc3mm** in the existing cluster.

6.2.1. Preparing the majority-maker node

- Install the node with the same operating system version as your HANA nodes.
- Configure the RHEL High Availability repository on the majority-maker node.
- If the **firewalld** service is installed and you are not using it on your hosts, disable the service. See Disabling the firewalld service.

6.2.2. Updating the host names in /etc/hosts

As for all cluster nodes, we recommend that you also add the majority-maker cluster node to the **/etc/hosts** file on each node. On the new **dc3mm** node you add all cluster nodes.

Procedure

- Add the new host to the **/etc/hosts** on all existing cluster nodes and add all hosts to the new **dc3mm** host, so that the resulting **/etc/hosts** entries are the same on all nodes:

```
[root]# cat /etc/hosts
...
192.168.100.101 dc1hana1.example.com dc1hana1
192.168.100.102 dc1hana2.example.com dc1hana2
192.168.100.103 dc2hana1.example.com dc2hana1
192.168.100.104 dc2hana2.example.com dc2hana2
192.168.100.110 dc3mm.example.com    dc3mm
```

Verification

- Check that you can ping the hosts between each other. This step is optional and an example only for a basic verification. The system resolves entries in **/etc/hosts** when you use the **ping** command:

```
[root]# ping dc3mm.example.com
```

```
PING dc3mm.example.com (192.168.100.110) 56(84) bytes of data.
64 bytes from dc3mm.example.com (192.168.100.110): icmp_seq=1 ttl=64 time=0.017 ms
…
```

Additional resources

hosts(5) man page on your system

6.2.3. Updating the cluster clone resources

The cluster automatically creates one copy of a clone resource for each cluster node and uses this to calculate resource allocations. For example, if the cluster consists of 4 HANA nodes and you add an additional majority-maker node for a quorum vote only, the cluster automatically calculates with all 5 cluster members for clone resource assignments. However, including this non-HANA node in the calculations can lead to unexpected impact when the cluster moves resources.

To prevent this influence you must configure the **clone-max** limit for all cloned resources explicitly to only the number of HANA nodes. Adjust the clone configuration before you add the new node to the cluster.

Procedure

1. Update the **SAPHanaTopology** resource clone with a limit to the number of HANA nodes, for example, **4**:

```
[root]# pcs resource update cln_SAPHanaTop_<SID>_HDB<instance> meta clone-max=4
```

2. Update the **SAPHanaController** resource clone with a limit to the number of HANA nodes, for example, **4**:

```
[root]# pcs resource update cln_SAPHanaCon_<SID>_HDB<instance> meta clone-max=4
```

3. Optional: If you have configured the **SAPHanaFilesystem** resource clone, then also update it with a limit to the number of HANA nodes, for example, **4**:

```
[root]# pcs resource update cln_SAPHanaFil_<SID>_HDB<instance> meta clone-max=4
```

Verification

- Check that the **clone-max** option is correct for all clone resources:

```
[root]# pcs resource config | grep -i clone
Clone: cln_SAPHanaTop_RH1_HDB02
   clone-max=4
   clone-node-max=1
Clone: cln_SAPHanaCon_RH1_HDB02
   clone-max=4
   clone-node-max=1
Clone: cln_SAPHanaFil_RH1_HDB02
   clone-max=4
   clone-node-max=1
```

- **clone-max** must be the number of HANA nodes, for example **4**. When this option is not displayed, it defaults to the total number of cluster nodes instead.

Additional resources

pcs(8) man page on your system

6.2.4. Installing the cluster components on the majority-maker node

Install the same cluster packages as on the existing cluster nodes to prepare the host in the same way.

Prerequisites

- You have configured the RHEL High Availability repository on the majority-maker host.

Procedure

1. Install the Red Hat High Availability Add-On software packages from the High Availability repository. Choose the same fence agents as you are using on the existing cluster nodes:

```
[root]# dnf install pcs pacemaker fence-agents-<model>
```

2. Start and enable the **pcsd** service on the new nodes. The **--now** parameter automatically starts the enabled service:

```
[root]# systemctl enable --now pcsd.service
```

3. Optional: If you use the local **firewalld** service you must enable the ports that are required by the Red Hat High Availability Add-On. Run this on the new node:

```
[root]# firewall-cmd --add-service=high-availability
[root]# firewall-cmd --runtime-to-permanent
```

4. Set a password for the user **hacluster** on the new node using the same password:

```
[root]# passwd hacluster
```

Verification

1. Check that the pcsd service is running and shows as **loaded** and **active** on the new node:

```
[root]# systemctl status pcsd.service
● pcsd.service - PCS GUI and remote configuration interface
   Loaded: loaded (/usr/lib/systemd/system/pcsd.service; enabled; preset: disabled)
   Active: active (running) since …
…
```

Additional resources

dnf(8) and **systemctl(1)** man pages on your system

6.2.5. Adding the new node to the cluster

Add the dedicated majority-maker node as a regular cluster member.

Prerequisites

- You have configured a cluster to which you want to add this node as a member.

Procedure

1. Authenticate the user **hacluster** for the new node in the cluster. Run this on one cluster node:

```
[root]# pcs host auth dc3mm
Username: hacluster
Password:
dc3mm: Authorized
```

   - Enter the node name with or without FQDN, as defined in the **/etc/hosts** file.
   - Enter the **hacluster** user password in the prompt.

2. Add the new node to the cluster. Run this on one cluster node:

```
[root]# pcs cluster node add dc3mm
No addresses specified for host 'dc3mm', using 'dc3mm'
Disabling sbd...
dc3mm: sbd disabled
Sending 'corosync authkey', 'pacemaker authkey' to 'dc3mm'
dc3mm: successful distribution of the file 'corosync authkey'
dc3mm: successful distribution of the file 'pacemaker authkey'
Sending updated corosync.conf to nodes...
dc2hana1: Succeeded
dc3mm: Succeeded
dc1hana2: Succeeded
dc2hana2: Succeeded
dc1hana1: Succeeded
dc2hana1: Corosync configuration reloaded
```

3. Add a location constraint to prevent any HANA resource from running on this node, and also prevent the cluster from trying to check the initial resource status. The following constraint definition uses a **regexp** expression to match all HANA resources. If required, adjust the pattern to match your resource names:

```
[root]# pcs constraint location add avoid-dc3mm \
regexp%.*SAPHana.* dc3mm -- -INFINITY resource-discovery=never
```

4. Enable the cluster on the new node to be started automatically on system start. Run on any node:

```
[root]# pcs cluster enable dc3mm
dc3mm: Cluster Enabled
```

5. Start the cluster on the new node:

```
[root]# pcs cluster start dc3mm
dc3mm: Starting Cluster...
```

Verification

1. **Check the location constraint that keeps HANA resources off the new node:**

```
[root]# pcs constraint location --full
Location Constraints:
…
  resource pattern '.*SAPHana.*' avoids node 'dc3mm' with score INFINITY (id: avoid-
dc3mm)
    resource-discovery=never
```

2. **Check the cluster status. Verify that the cluster daemon services are in the desired state. Run this on the new node to also verify the local daemon status at the end:**

```
[root]# pcs status --full
…

Node List:
  * Node dc1hana1 (1): online, feature set 3.19.6
  * Node dc1hana2 (2): online, feature set 3.19.6
  * Node dc2hana1 (3): online, feature set 3.19.6
  * Node dc2hana2 (4): online, feature set 3.19.6
  * Node dc3mm (5): online, feature set 3.19.6

Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]:
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc1hana2
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc1hana1
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc2hana1
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc2hana2
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02]
(promotable):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc1hana2
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Promoted
dc1hana1
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc2hana1
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc2hana2
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02]:
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc1hana2
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc1hana1
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc2hana1
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
```

```
dc2hana2
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPAddr2):        Started dc1hana1
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPAddr2):        Started dc2hana1

…

PCSD Status:
  dc1hana1: Online
  dc1hana2: Online
  dc2hana1: Online
  dc2hana2: Online
  dc3mm: Online

Daemon Status:
  corosync: active/enabled
  pacemaker: active/enabled
  pcsd: active/enabled
```

- The new node must be displayed in the **Node List** and in the **PCSD Status** list.
- The new node must not show a status in the **Full List of Resources**.

3. Verify the quorum status. The new node adds a vote, so that there is now an odd number of votes and there can be no equal 50/50 split anymore:

```
[root]# pcs quorum status
Quorum information
------------------
Date:             Tue Sep  2 14:56:29 2025
Quorum provider:  corosync_votequorum
Nodes:            5
Node ID:          5
Ring ID:          1.27
Quorate:          Yes

Votequorum information
----------------------
Expected votes:   5
Highest expected: 5
Total votes:      5
Quorum:           3
Flags:            Quorate

Membership information
----------------------
    Nodeid      Votes    Qdevice Name
         1          1         NR dc1hana1
         2          1         NR dc1hana2
         3          1         NR dc2hana1
         4          1         NR dc2hana2
         5          1         NR dc3mm (local)
```

Next steps

- Add the new node to your individual fencing method. See Configuring fencing in a Red Hat High Availability cluster.

Additional resources

- Managing cluster nodes
- **pcs(8)** man page on your system

CHAPTER 7. TESTING THE SETUP

Test your new HANA HA cluster thoroughly before you enable it for production workloads.

Enhance the basic example test cases with your specific requirements.

7.1. DETECTING THE SYSTEM REPLICATION STATE CHANGES

You must monitor the sync state information in logs and cluster attributes when disrupting the system replication, to test the correct functionality of the **HanaSR** HA/DR provider.

In this test, you use the primary site for monitoring the system replication status and for verifying the log messages. On a secondary instance you freeze the indexserver process to simulate a system replication issue while the primary remains fully intact.

Prerequisites

- You have configured the mandatory **HanaSR** HA/DR provider.
- Your HANA instances are in a healthy state on all cluster nodes and the system replication is in sync.

Procedure

1. As user **<sid>adm** go to the HANA Python directory on the primary site and check the current system replication state. Verify that it is **ACTIVE** and fully synced:

```
rh1adm $ cdpy; python systemReplicationStatus.py
…
status system replication site "2": ACTIVE
overall system replication status: ACTIVE
…
```

2. Verify that the **srHook** and **srPoll** cluster attributes are both **SOK** in the attributes summary of the secondary site. Run this command as the **root** user on any node in a separate terminal to keep track of the attribute changes:

```
[root]# watch SAPHanaSR-showAttr
...
Site lpt       lss mns      opMode    srHook srMode srPoll srr
-----------------------------------------------------------------
DC2  30         4   dc2hana1 logreplay SOK    sync   SOK    S
DC1  1757076772 4   dc1hana1 logreplay PRIM   sync   PRIM   P
...
```

   You can use the **watch** command to run the command in a loop at a default interval of 2 seconds.

3. On an instance on the secondary site, for example, **dc2hana2**, get the process ID (**PID**) of the **hdbindexserver** process. For example, you can get it from the **PID** column of the **HDB info** output as user **<sid>adm**:

```
rh1adm $ HDB info
```

4. On the same instance on the secondary site, use the **PID** to simulate a hanging **hdbindexserver** process by sending the **STOP** signal to the process. This freezes the process and blocks it from communicating and syncing the instance between the nodes:

```
rh1adm $ kill -STOP <PID>
```

Verification

1. On the primary site, watch the system replication status for the change on any primary instance. In the following example the system's **cut** utility helps you limit the output to certain fields for readability. Remove it to see all columns of the table formatted text output. In the example we froze the indexserver on the secondary node **dc2hana2**, which results in a replication error with that node's counterpart on the primary site, **dc1hana2**:

```
rh1adm $ cdpy; watch "python systemReplicationStatus.py | cut -d '|' -f 1-3,5,9,13-"
...
|Database |Host     |Service Name |Secondary |Secondary     |Replication |Replication
|Replication                  |Secondary    |
|         |         |             |Host      |Active Status |Mode        |Status      |Status Details
|Fully Synced |
|-------- |-------- |------------ |--------- |------------- |----------- |----------- |----------------------------
|------------ |
|RH1      |dc1hana2 |indexserver  |dc2hana2  |YES           |SYNC        |ERROR       |Log
shipping timeout occurred |       False |
|SYSTEMDB |dc1hana1 |nameserver   |dc2hana1  |YES           |SYNC        |ACTIVE      |
|       True |
|RH1      |dc1hana1 |xsengine     |dc2hana1  |YES           |SYNC        |ACTIVE      |
|       True |
|RH1      |dc1hana1 |indexserver  |dc2hana1  |YES           |SYNC        |ACTIVE      |
|       True |

status system replication site "2": ERROR
overall system replication status: ERROR
...
```

The replication status changes to **ERROR** for the indexserver service after a bit. It can take a while to react on an idle instance, wait a minute or more.

2. On the primary site's master name server node, check the HANA nameserver process log for the related messages as the **<sid>adm** user:

```
rh1adm $ cdtrace; grep -he 'HanaSR.srConnectionChanged.*' nameserver_*
ha_dr_HanaSR     HanaSR.py(00056) : HanaSR 1.001.1 HanaSR.srConnectionChanged
method called with Dict={'hostname': 'dc1hana2', 'port': '30003', 'volume': 4, 'service_name':
'indexserver', 'database': 'RH1', 'status': 11, 'database_status': 11, 'system_status': 11,
'timestamp': '2025-09-12T11:15:08.003728+00:00', 'is_in_sync': False, 'system_is_in_sync':
False, 'reason': '', 'siteName': 'DC2'}
ha_dr_HanaSR     HanaSR.py(00065) : HanaSR HanaSR.srConnectionChanged
system_status=11 SID=RH1 in_sync=False reason=
ha_dr_HanaSR     HanaSR.py(00091) : HanaSR.srConnectionChanged() CALLING CRM:
<sudo /usr/sbin/crm_attribute -n hana_rh1_site_srHook_DC2  -v SFAIL -t crm_config -s
SAPHanaSR> ret_code=
```

The nameserver process log contains the event that HANA triggers with details. It also includes the **sudo** command that the **HanaSR** hook script runs to update the **srHook** cluster attribute.

3. Verify that both cluster attributes for the system replication status, **srHook** and **srPoll**, show the **SFAIL** status of the secondary site. Run the following as the **root** user on any HANA node or use the open terminal from the previous steps to watch the changes:

```
[root]# SAPHanaSR-showAttr
...
Site lpt       lss mns      opMode    srHook srMode srPoll srr
-----------------------------------------------------------------
DC2  10         4   dc2hana1 logreplay SFAIL  sync   SFAIL  S
DC1  1757079061 4   dc1hana1 logreplay PRIM   sync   PRIM   P
...
```

4. Unblock the previously frozen **hdbindexserver** PID to enable it again. Run this on the secondary instance on which you blocked the **hdbindexserver** process for the test:

```
rh1adm $ kill -CONT <PID>
```

5. Repeat the previous checks to verify that the system replication recovers fully after a bit. The cluster does not trigger any actions during this test since the resources remain running. Ensure that the system replication status is healthy again, fully synced and the cluster attributes are set to **SOK** again for the secondary site.

Additional resources

watch(1), **cut(1)** and **kill(1)** man pages on your system

7.2. TRIGGERING THE INDEXSERVER CRASH RECOVERY

Test the functionality of the **ChkSrv** HA/DR provider by simulating the crash of an **hdbindexserver** process. You can run this on the primary or on the secondary site. The exact recovery actions depend on the overall configuration. The following steps demonstrate the activity when using **action_on_lost = stop** in the hook configuration..

Prerequisites

- You have configured the **ChkSrv** HA/DR provider. Skip this test if you have not configured this optional hook.
- Your HANA instances have a healthy HANA system replication.
- You have no failures in the cluster status.

Procedure

1. Use a separate terminal to monitor the HANA processes as user **<sid>adm** on the instance on which you run this test:

```
rh1adm $ watch "sapcontrol -nr ${TINSTANCE} -function GetProcessList | column -s ',' -t"
```

2. **In another terminal on the same HANA instance, kill the hdbindexserver process:**

```
rh1adm $ kill <PID>
```

Verification

1. **Check the dedicated HANA nameserver trace log on the same instance and identify the event and related action, as user <sid>adm:**

```
rh1adm $ cdtrace; less nameserver_chksrv.trc
...
ChkSrv version 1.001.1. Method srServiceStateChanged method called.
ChkSrv srServiceStateChanged method called with Dict={'hostname': 'dc2hana2',
 'service_name': 'indexserver', 'service_port': '30203', 'service_status': 'stopping',
 'service_previous_status': 'yes', 'timestamp': '2025-09-15T15:07:09.353198+00:00',
 'daemon_status': 'yes', 'database_id': '3', 'database_name': 'RH1',
 'database_status': 'yes', 'details': ''}
ChkSrv srServiceStateChanged method called with SAPSYSTEMNAME=RH1
srv:indexserver-30203-stopping-yes db:RH1-3-yes daem:yes
LOST: indexserver event looks like a lost indexserver (status=stopping)
LOST: stop instance. action_on_lost=stop
...
```

2. **Check the cluster status for resource failure information on any cluster node, as user root:**

```
[root]# pcs status --full
...

Failed Resource Actions:
  * rsc_SAPHanaCon_RH1_HDB02_monitor_61000 on dc2hana1 'not running' (7): call=26,
status='complete', ...

...
```

3. **Check the system log for the related cluster actions on the on the test node, for example, dc2hana2, as user root:**

```
[root]# grep rsc_SAPHanaCon_RH1_HDB02 /var/log/messages
...
Sep 15 15:08:17 dc2hana1 pacemaker-controld[17045]: notice: Result of monitor operation
for rsc_SAPHanaCon_RH1_HDB02 on dc2hana1: not running
Sep 15 15:08:17 dc2hana1 pacemaker-controld[17045]: notice:
rsc_SAPHanaCon_RH1_HDB02_monitor_61000@dc2hana1 output [ 10 ]
Sep 15 15:08:17 dc2hana1 pacemaker-controld[17045]: notice: Transition 32 action 29
(rsc_SAPHanaCon_RH1_HDB02_monitor_61000 on dc2hana1): expected 'ok' but got 'not
running'
Sep 15 15:08:17 dc2hana1 pacemaker-attrd[17043]: notice: Setting last-failure-
rsc_SAPHanaCon_RH1_HDB02#monitor_61000[dc2hana1] in instance_attributes: (unset) ->
1757948897
Sep 15 15:08:17 dc2hana1 pacemaker-attrd[17043]: notice: Setting fail-count-
rsc_SAPHanaCon_RH1_HDB02#monitor_61000[dc2hana1] in instance_attributes: (unset) ->
1
Sep 15 15:08:17 dc2hana1 pacemaker-schedulerd[17044]: warning: Unexpected result (not
running) was recorded for monitor of rsc_SAPHanaCon_RH1_HDB02:2 on dc2hana1 at Sep
```

```
15 15:08:17 2025
Sep 15 15:08:17 dc2hana1 pacemaker-schedulerd[17044]: notice: Actions: Recover
rsc_SAPHanaCon_RH1_HDB02:2     (            Unpromoted dc2hana1 )
Sep 15 15:08:17 dc2hana1 pacemaker-schedulerd[17044]: warning: Unexpected result (not
running) was recorded for monitor of rsc_SAPHanaCon_RH1_HDB02:2 on dc2hana1 at Sep
15 15:08:17 2025
Sep 15 15:08:17 dc2hana1 pacemaker-schedulerd[17044]: notice: Actions: Recover
rsc_SAPHanaCon_RH1_HDB02:2     (            Unpromoted dc2hana1 )
Sep 15 15:08:17 dc2hana1 pacemaker-controld[17045]: notice: Initiating stop operation
rsc_SAPHanaCon_RH1_HDB02_stop_0 locally on dc2hana1
Sep 15 15:08:17 dc2hana1 pacemaker-controld[17045]: notice: Requesting local execution
of stop operation for rsc_SAPHanaCon_RH1_HDB02 on dc2hana1
...
```

The next **SAPHanaController** resource monitor reports the unexpectedly stopped HANA instance as a failure and initiates the recovery steps according to the configuration. If **PREFER_SITE_TAKEOVER** is enabled and you executed the test on a primary instance, it triggers a HANA takeover to the secondary site.

Next steps

- When necessary, depending on the configuration, manually reregister the stopped former primary HANA site and start it using HANA tools. For more information, refer to Registering the former primary HANA site as a secondary HANA site after a takeover.
- Clear any failure notifications from the cluster that might be there from previous testing. For more information see Cleaning up the failure history.

Additional resources

watch(1) and **kill(1)** man pages on your system

7.3. TRIGGERING A HANA TAKEOVER USING CLUSTER COMMANDS

Use the cluster command to move the promoted resource to the other site and manually test the planned takeover of the primary to the secondary site.

Prerequisites

- Your HANA instances have a healthy HANA system replication.
- You have no failures in the cluster status.

Procedure

- Switch the primary site to the secondary site. Run the cluster command as user **root** on any node:

```
[root]# pcs resource move cln_SAPHanaCon_<SID>_HDB<instance>
Location constraint to move resource 'cln_SAPHanaCon_RH1_HDB02' has been created
Waiting for the cluster to apply configuration changes...
Location constraint created to move resource 'cln_SAPHanaCon_RH1_HDB02' has been
removed
Waiting for the cluster to apply configuration changes...
resource 'cln_SAPHanaCon_RH1_HDB02' is promoted on node 'dc2hana1'
```

Verification

- Verify that the **SAPHanaController** resource is now promoted on the other site:

```
[root]# pcs resource status cln_SAPHanaCon_RH1_HDB02
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02]
(promotable):
    * Promoted: [ dc2hana1 ]
    * Unpromoted: [ dc1hana2 dc2hana2 ]
    * Stopped: [ dc1hana1 ]
```

  The status of the previous primary instance depends on the **AUTOMATED_REGISTER** parameter of the **SAPHanaController** resource. The instance stops until manual intervention when **AUTOMATED_REGISTER** is **false**, otherwise the instance restarts automatically and reregisters as the new secondary instance.

Next steps

- When necessary, depending on the configuration, manually reregister the stopped former primary HANA site and start it using HANA tools. For more information, refer to Registering the former primary HANA site as a secondary HANA site after a takeover.
- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.

Additional resources

- Is it possible to use SAP HANA "Takeover with Handshake" option with the HA Solutions for managing HANA System Replication?
- **pcs(8)** man page on your system

7.4. TRIGGERING THE SAPHANAFILESYSTEM FAILURE ACTION

Block the write access to the monitored directory to test the correct behavior of the **SAPHanaFilesystem** resource. You can test this on any instance. Only a primary instance triggers a failure and recovery action. On a secondary node the resource does not trigger an action.

Prerequisites

- You have configured the **SAPHanaFilesystem** resource. Skip this test if you have not configured this optional resource.

Procedure

1. Create a temporary file on a local filesystem of the node you want to test:

```
[root]# touch /tmp/test
```

2. Set the local file to be immutable, which prevents write access:

```
[root]# chattr +i /tmp/test
```

3. Go to the hidden directory which the **SAPHanaFilesystem** resource uses to test read and write filesystem access and enter the node you want to test:

```
[root]# cd /hana/shared/<SID>/.heartbeat_SAPHanaFilesystem/<node>
```

4. Change the **test** file, which the**SAPHanaFilesystem** resource creates, to become a symbolic link that points to the temporary local file. Since the temporary target test file cannot be modified, the resource fails after the next monitor cycle:

```
[root]# ln -sf /tmp/test test
```

NFS filesystems do not support extended attributes by default. The symbolic link bridges this gap for the test.

5. Verify the behavior during the simulated failure.

   a. You can check the **/var/log/messages** file for the related log message if the resource action is set to **ignore**:

```
[root]# grep -e 'SAPHanaFil.*ON_FAIL_ACTION' /var/log/messages
... SAPHanaFilesystem(rsc_SAPHanaFil_RH1_HDB02)[715184]: INFO: -2- RA monitor()
ON_FAIL_ACTION=ignore => ignore FS error, do not create poison pill file
```

   b. If the resource action is set to **fence** you can observe the fencing action:

```
[root]# pcs status --full
...

Failed Resource Actions:
  * rsc_SAPHanaFil_RH1_HDB02_stop_0 on dc1hana1 'error' (1): ...

Pending Fencing Actions:
  * reboot of dc1hana1 pending: client=pacemaker-controld.1694, origin=dc1hana2
```

6. You must remove the blocker again after the test. If the node is fenced, delete the symbolic link after the node is running again. The resource creates the regular test file again during the next check:

```
[root]# rm -f /hana/shared/<SID>/.heartbeat_SAPHanaFilesystem/<node>/test
```

7. Clean up the temporary local test file:

```
[root]# chattr -i /tmp/test; rm -f /tmp/test
```

Next steps

- When necessary, depending on the configuration, manually reregister the stopped former primary HANA site and start it using HANA tools. For more information, refer to Registering the former primary HANA site as a secondary HANA site after a takeover.
- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.

Additional resources

chattr(1), **ln(1)** and **pcs(8)** man pages on your system

7.5. CRASHING THE NODE WITH A PRIMARY INSTANCE

Simulate the crash of the cluster node on which a primary instance is running to test the behavior of your HANA cluster resources.

Prerequisites

- Your HANA instances have a healthy HANA system replication.
- You have no failures in the cluster status.

Procedure

- Trigger a crash on a HANA node on the primary site. This command immediately causes a crash of the node with no further warning:

```
[root]# echo c > /proc/sysrq-trigger
```

Verification

- The cluster detects the failed node and fences it. You can watch the cluster activity on any of the remaining nodes:

```
[root]# pcs status --full
...
Pending Fencing Actions:
  * reboot of dc1hana1 pending: client=pacemaker-controld.1685, origin=dc1hana2
...
```

- The secondary site takes over and becomes promoted as the new primary.
- The fenced former primary node recovers according to your fencing and **SAPHanaController** resource configuration.

Next steps

- When necessary, depending on the configuration, manually reregister the stopped former primary HANA site and start it using HANA tools. For more information, refer to Registering the former primary HANA site as a secondary HANA site after a takeover.
- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.

Additional resources

How to use the SysRq Facility to collect information from a RHEL system

7.6. CRASHING THE NODE WITH A SECONDARY INSTANCE

Simulate the crash of the cluster node on which a secondary instance is running to test the behavior of your HANA cluster resources.

Procedure

- Trigger a crash of a HANA node on the secondary site. This command immediately causes a crash of the node with no further warning:

```
[root]# echo c > /proc/sysrq-trigger
```

Verification

- The cluster detects the failed node and fences it. You can watch the cluster activity on any of the remaining nodes:

```
[root]# pcs status --full
...
Pending Fencing Actions:
  * reboot of dc2hana1 pending: client=pacemaker-controld.1694, origin=dc1hana1
...
```

- The primary site remains running while the secondary node restarts and recovers. The fenced node recovery depends on your fencing configuration.

Next steps

- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.

Additional resources

How to use the SysRq Facility to collect information from a RHEL system

7.7. STOPPING THE PRIMARY SITE USING SAP COMMANDS

Test the behavior of the cluster when you manage the primary HANA site outside of the cluster using HANA commands.

Since the cluster is not aware of the execution of HANA commands, it detects the change as a failure and triggers the configured recovery actions.

Prerequisites

- Your HANA instances have a healthy HANA system replication.
- You have no failures in the cluster status.

Procedure

- Stop the primary HANA site as the **<sid>adm** user outside of the cluster. Run on one HANA instance on the primary site:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function StopSystem HDB
```

Verification

- The cluster notices the stopped instance as a failure and initiates the recovery of the primary site:

```
[root]# pcs status --full
...
Migration Summary:
  * Node: dc1hana1 (1):
    * rsc_SAPHanaCon_RH1_HDB02: migration-threshold=5000 fail-count=1 last-failure=...

Failed Resource Actions:
  * rsc_SAPHanaCon_RH1_HDB02_monitor_61000 on dc1hana1 'not running' ...
```

  If you configured and enabled both the **PREFER_SITE_TAKEOVER** and **AUTOMATED_REGISTER** parameters in the**SAPHanaController** resource, the cluster triggers a HANA takeover to the secondary site and automatically registers the failed primary as the new secondary. Otherwise it recovers the failed primary according to your configuration.

Next steps

- When necessary, depending on the configuration, manually reregister the stopped former primary HANA site and start it using HANA tools. For more information, refer to Registering the former primary HANA site as a secondary HANA site after a takeover.
- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.

7.8. STOPPING THE SECONDARY SITE USING SAP COMMANDS

Test the behavior of the cluster when you manage the secondary HANA site outside of the cluster using HANA commands.

Since the cluster is not aware of the execution of HANA commands, it detects the change as a failure and triggers the configured recovery actions.

Prerequisites

- You have no failures in the cluster status.

Procedure

- Stop the secondary HANA site as the **<sid>adm** user outside of the cluster. Run on one HANA instance on the secondary site:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function StopSystem HDB
```

Verification

- The cluster notices the stopped instance as a failure and recovers the secondary site:

```
[root]# pcs status --full
...
```

```
Migration Summary:
  * Node: dc2hana1 (2):
    * rsc_SAPHanaCon_RH1_HDB02: migration-threshold=5000 fail-count=1 last-failure=...

Failed Resource Actions:
  * rsc_SAPHanaCon_RH1_HDB02_monitor_61000 on dc2hana1 'not running' ...
```

Next steps

- **Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.**

CHAPTER 8. FINISHING THE SETUP

Ensure that the final setup is complete and the systems and resources are healthy, then you can enable the environment for production workloads.

8.1. ENABLING THE AUTOMATIC REGISTRATION OF HANA AFTER A TAKEOVER

If you want a previously failed primary site to automatically recover as a fully functional secondary site without manual verification of the data consistency, you can enable the **SAPHanaController** resource to re-register the site right after a takeover.

This enables the previously failed primary site to continue the HANA system replication and automatically take over again in the event of a new failure of the new primary site.

Your HANA operator must decide if they first require to manually check the health of the previously failed instance and re-register the HANA site afterwards, or if the priority is on a faster automatic recovery of the full high availability.

Procedure

- Update the **SAPHanaController** resource and override the default **AUTOMATED_REGISTER**:

```
[root]# pcs resource update rsc_SAPHanaCon_<SID>_HDB<instance>
AUTOMATED_REGISTER=true
```

Verification

- Check that **AUTOMATED_REGISTER** is set to**true**:

```
[root]# pcs resource config rsc_SAPHanaCon_RH1_HDB02
Resource: rsc_SAPHanaCon_RH1_HDB02 (class=ocf provider=heartbeat
type=SAPHanaController)
  Attributes: rsc_SAPHanaCon_RH1_HDB02-instance_attributes
    AUTOMATED_REGISTER=true
    DUPLICATE_PRIMARY_TIMEOUT=7200
    InstanceNumber=02
    PREFER_SITE_TAKEOVER=true
    SID=RH1
...
```

> **WARNING**
>
> Setting **AUTOMATED_REGISTER** to **true** can potentially increase the risk of data loss or corruption. If the HA cluster triggers a takeover when the data on the secondary HANA site is not fully in sync, the automatic registration of the old primary HANA site as the new secondary HANA site results in data loss on this site and any data that was not synced before the takeover occurred is lost as well.
>
> For more information, see the article on the SAP Technology Blog for Members: Be Prepared for Using Pacemaker Cluster for SAP HANA – Part 2: Failure of Both Nodes.

Additional resources

pcs(8) man page on your system

8.2. REVIEWING THE FINAL CLUSTER STATE

After the configuration of a 4-node cluster for a scale-out HANA system replication setup, the status looks like in the below example.

Your cluster state may deviate from the example, depending on your setup of optional or platform dependent resources, like the individual fencing or VIP resources.

Also, you can decide if you want to disable the cluster service, so that it does not start automatically on system boot. This requires manual intervention on every system boot, but allows you more control and supervision for the startup.

```
[root]# pcs status --full
Cluster name: hana-scaleout-cluster
Cluster Summary:
  * Stack: corosync (Pacemaker is running)
  * Current DC: dc2hana1 (3) (version 2.1.9-1.2.el9_6-49aab9983) - partition with quorum
  * Last updated: Mon Sep 15 16:10:09 2025 on dc1hana2
  * Last change:  Mon Sep 15 16:10:01 2025 by root via root on dc1hana1
  * 4 nodes configured
  * 15 resource instances configured

Node List:
  * Node dc1hana1 (1): online, feature set 3.19.6
  * Node dc1hana2 (2): online, feature set 3.19.6
  * Node dc2hana1 (3): online, feature set 3.19.6
  * Node dc2hana2 (4): online, feature set 3.19.6

Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]:
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):      Started dc1hana1
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):      Started dc1hana2
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):      Started dc2hana1
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):      Started dc2hana2
```

```
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02] (promotable):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Promoted dc1hana1
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted dc1hana2
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted dc2hana1
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted dc2hana2
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02]:
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started dc1hana1
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started dc1hana2
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started dc2hana1
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started dc2hana2
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):        Started dc1hana1
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):        Started dc2hana1

Node Attributes:
  * Node: dc1hana1 (1):
    * hana_rh1_clone_state              : PROMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC1
    * hana_rh1_sra                      : -
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc1hana1
    * master-rsc_SAPHanaCon_RH1_HDB02   : 150
  * Node: dc1hana2 (2):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : slave:slave:worker:slave
    * hana_rh1_site                     : DC1
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc1hana2
    * master-rsc_SAPHanaCon_RH1_HDB02   : -10000
  * Node: dc2hana1 (3):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC2
    * hana_rh1_sra                      : -
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc2hana1
    * master-rsc_SAPHanaCon_RH1_HDB02   : 100
  * Node: dc2hana2 (4):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : slave:slave:worker:slave
    * hana_rh1_site                     : DC2
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc2hana2
    * master-rsc_SAPHanaCon_RH1_HDB02   : -12200

Migration Summary:

Tickets:

PCSD Status:
  dc1hana1: Online
  dc1hana2: Online
```

```
  dc2hana1: Online
  dc2hana2: Online

Daemon Status:
  corosync: active/enabled
  pacemaker: active/enabled
  pcsd: active/enabled
```

In a healthy setup the additional cluster attributes appear like in this example:

```
[root]# SAPHanaSR-showAttr
Global cib-update dcid prim sec sid topology
--------------------------------------------
global 0.254.0    3    DC1  DC2 RH1 ScaleOut

Resource                 promotable
-----------------------------------
cln_SAPHanaCon_RH1_HDB02 true
cln_SAPHanaTop_RH1_HDB02

Site lpt        lss mns      opMode    srHook srMode srPoll srr
-----------------------------------------------------------------
DC2  30         4   dc2hana1 logreplay SOK    sync   SOK    S
DC1  1757952664 4   dc1hana1 logreplay PRIM   sync   PRIM   P

Host     clone_state roles                        score  site sra srah version     vhost
-----------------------------------------------------------------------------------------
dc1hana1 PROMOTED    master1:master:worker:master 150    DC1  -   -    2.00.083.00 dc1hana1
dc1hana2 DEMOTED     slave:slave:worker:slave     -10000 DC1      -    2.00.083.00 dc1hana2
dc2hana1 DEMOTED     master1:master:worker:master 100    DC2  -   -    2.00.083.00 dc2hana1
dc2hana2 DEMOTED     slave:slave:worker:slave     -12200 DC2      -    2.00.083.00 dc2hana2
```

CHAPTER 9. MAINTENANCE PROCEDURES

You must apply specific steps to ensure that the cluster does not cause unplanned impact so that you can perform maintenance of different components of SAP HANA system replication HA environments.

Use maintenance procedures to keep your cluster in a healthy state during planned change activity or to restore the health after unplanned incidents.

9.1. CLEANING UP THE FAILURE HISTORY

Clear any failure notifications from the cluster that may be there from previous testing. This resets the failure counters and the migration thresholds.

Procedure

1. Clean up resource failures:

```
[root]# pcs resource cleanup
```

2. Clean up the STONITH failure history:

```
[root]# pcs stonith history cleanup
```

Verification

- Check the overall cluster status and confirm that no failures are displayed anymore:

```
[root]# pcs status --full
```

- Check that the stonith history for fencing actions has 0 events:

```
[root]# pcs stonith history
```

Additional resources

pcs(8) man page on your system

9.2. TRIGGERING A HANA TAKEOVER USING CLUSTER COMMANDS

Use the cluster control to execute a simple takeover of the primary site to the secondary site.

For detailed steps, refer to the section Testing the setup - Triggering a HANA takeover using cluster commands.

9.3. UPDATING THE OPERATING SYSTEM AND HA CLUSTER COMPONENTS

For updates or offline changes on the HA cluster, the operating system or even the system hardware, you must follow the Recommended Practices for Applying Software Updates to a RHEL High Availability or Resilient Storage Cluster.

9.4. PERFORMING MAINTENANCE ON THE SAP HANA INSTANCES

For any kind of maintenance of applications or other components that the HA cluster manages, you must enable the cluster **maintenance** mode to prevent the cluster from any interference during the maintenance.

During the update of your HANA instances, the cluster remains running, but is not actively monitoring resources or taking any actions. After the change on the HANA instances is done, it is vital to refresh the cluster resource status and verify that the detected resource states are all correct. Only then you can safely disable the maintenance mode without unexpected cluster actions.

If you need to stop the cluster for the maintenance activity, ensure that you set **maintenance** mode first, then stop and start the cluster on the node as required for the HANA maintenance.

Prerequisites

- You have configured the Pacemaker cluster to manage the HANA system replication.

Procedure

1. Set **maintenance** mode for the entire cluster:

```
[root]# pcs property set maintenance-mode=true
```

   Setting maintenance for the whole cluster ensures that no activity during the maintenance phase can trigger cluster actions and impact the HANA update process.

2. Verify that the cluster resource management is fully disabled:

```
[root]# pcs status
...
              *** Resource management is DISABLED ***
  The cluster will not attempt to start, stop or recover services

Node List:
  * Online: [ dc1hana1 dc1hana2 dc2hana1 dc2hana2 ]

Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]
(maintenance):
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc1hana1 (maintenance)
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc1hana2 (maintenance)
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc2hana1 (maintenance)
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc2hana2 (maintenance)
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02]
(promotable, maintenance):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Promoted
dc1hana1 (maintenance)
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc1hana2 (maintenance)
```

```
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc2hana1 (maintenance)
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc2hana2 (maintenance)
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02] (maintenance):
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc1hana1 (maintenance)
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc1hana2 (maintenance)
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc2hana1 (maintenance)
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc2hana2 (maintenance)
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):        Started dc1hana1
(maintenance)
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):        Started dc2hana1
(maintenance)

...
```

3. Update the HANA instances using the SAP procedure. If you have to perform a takeover during the HANA update, you can use the SAP HANA Takeover with Handshake option. For more information see also Is it possible to use SAP HANA "Takeover with Handshake" option with the HA Solutions for managing HANA System Replication?.
   If you stop the cluster in this step, ensure that you start it again before you proceed with the next steps. Keep the maintenance mode enabled.

4. After the HANA update, verify that the HANA system replication is working correctly. Use the **systemReplicationStatus.py** script to show the status of the HANA system replication on the primary site. Below is the example after a manual takeover to the secondary site during the maintenance:

```
[root]# su - <sid>adm -c "HDBSettings.sh systemReplicationStatus.py \
--sapcontrol=1 | grep -i replication_status="
service/dc2hana2/30003/REPLICATION_STATUS=ACTIVE
service/dc2hana1/30001/REPLICATION_STATUS=ACTIVE
service/dc2hana1/30007/REPLICATION_STATUS=ACTIVE
service/dc2hana1/30003/REPLICATION_STATUS=ACTIVE
site/1/REPLICATION_STATUS=ACTIVE
overall_replication_status=ACTIVE
```

   Before you proceed, ensure that the system replication is healthy and reported as **ACTIVE**.

5. Refresh all cluster resources to execute one monitor operation and update their status:

```
[root]# pcs resource refresh
Waiting for 1 reply from the controller
... got reply (done)
```

   It is crucial that the HANA resources update cluster and node attributes to reflect the new HANA system replication status. It ensures that the cluster has the correct information and does not trigger recovery actions due to incorrect status information, after the maintenance stops.

6. **Check the cluster status and verify the resource status and main HANA resource score attribute. All resources must show as Started and the promotable resources must show as Unpromoted on all nodes:**

```
[root]# pcs status resources
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]
(maintenance):
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc1hana1 (maintenance)
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc1hana2 (maintenance)
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc2hana1 (maintenance)
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started
dc2hana2 (maintenance)
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02]
(promotable, maintenance):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc1hana1 (maintenance)
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc1hana2 (maintenance)
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc2hana1 (maintenance)
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
dc2hana2 (maintenance)
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02] (maintenance):
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc1hana1 (maintenance)
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc1hana2 (maintenance)
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc2hana1 (maintenance)
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started
dc2hana2 (maintenance)
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):        Started dc1hana1
(maintenance)
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):        Started dc2hana1
(maintenance)
```

7. **Check the cluster attributes and verify that srHook, roles and score attributes are in the correct new state:**

```
[root]# SAPHanaSR-showAttr
...

Site lpt        lss mns      opMode    srHook srMode srPoll srr
-----------------------------------------------------------------
DC2  1757953883 4   dc2hana1 logreplay PRIM   sync   SOK    S
DC1  30         4   dc1hana1 logreplay SOK    sync   PRIM   P

Host     clone_state roles                        score  site sra srah version     vhost
-----------------------------------------------------------------------------------------
dc1hana1 DEMOTED     master1:master:worker:master 100    DC1  -   -    2.00.083.00
dc1hana1
dc1hana2 DEMOTED     slave:slave:worker:slave     -12200 DC1      -    2.00.083.00
dc1hana2
```

```
dc2hana1 DEMOTED     master1:master:worker:master 150    DC2  -   -    2.00.083.00
dc2hana1
dc2hana2 DEMOTED     slave:slave:worker:slave     -10000 DC2      -    2.00.083.00
dc2hana2
```

- The **srHook** is **PRIM** on the node that is running the primary instance and it shows **SOK** on the correct secondary.
- The **srPoll** attribute still shows the previous status. It will update after you remove the maintenance.
- The **score** is **150** on the node where the main primary is running and **100** on the main secondary.

8. When the checks of the previous steps show the landscape in the expected healthy state, you can remove the maintenance mode of the cluster again:

```
[root]# pcs property set maintenance-mode=
```

When you lift the maintenance it triggers a monitor run of all resources again. The cluster updates the status of the promotable resources to **Promoted** and **Unpromoted** in correspondence to the location of the primary and secondary instances. The resources now also update the **srPoll** attribute again to match the **srHook** attribute value.

Additional resources

- Update SAP HANA Systems Running in a System Replication Setup
- **pcs(8)** man page on your system

9.5. REGISTERING THE FORMER PRIMARY HANA SITE AS A SECONDARY HANA SITE AFTER A TAKEOVER

When you configure **AUTOMATED_REGISTER=false** in the **SAPHanaController** resource, which is the default, you must manually register the former primary site as the new secondary after takeover and start it. Otherwise, the unregistered site remains stopped.

Procedure

1. Register the former primary site as the new secondary site. Run as user **<sid>adm** on one stopped former primary instance:

```
rh1adm $ hdbnsutil -sr_register --remoteHost=<node> \
--remoteInstance=${TINSTANCE} --replicationMode=sync \
--operationMode=logreplay --name=<site>
```

- Replace **<node>** with the new primary instance host, for example **dc2hana1** if there was a takeover from dc1hana1 to dc2hana1.
- Replace **<site>** with your new secondary HANA site name, for example **DC1** if dc1hana1 is to be registered as a secondary.
- Choose the values for **replicationMode** and **operationMode** according to your requirements for the system replication.

- **$TINSTANCE** is an environment variable that is set automatically for user **<sid>adm** by reading the HANA instance profile. The variable value is the HANA instance number.

2. Start the secondary HANA site. Run as **<sid>adm** on one new secondary instance node:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function StartSystem HDB
```

Verification

- On one new primary instance, show the current status of the re-established HANA system replication. Below is the example after a takeover from **dc1hana1** to **dc2hana1** and DC2 is the new primary site:

```
rh1adm $ cdpy; python systemReplicationStatus.py
|Database |Host     |Port  |Service Name |Volume ID |Site ID |Site Name |Secondary
|Secondary |Secondary |Secondary |Secondary     |Replication |Replication |Replication
|Secondary    |
|         |         |      |             |          |        |          |Host      |Port      |Site ID   |Site Name |Active
Status |Mode        |Status      |Status Details |Fully Synced |
|-------- |-------- |----- |------------ |--------- |------- |--------- |--------- |--------- |--------- |--
----------- |----------- |----------- |-------------- |------------ |
|RH1      |dc2hana2 |30003 |indexserver  |        4 |      2 |DC2       |dc1hana2  |    30003 |        1
|DC1       |YES           |SYNC        |ACTIVE      |              |        True |
|SYSTEMDB |dc2hana1 |30001 |nameserver   |        1 |      2 |DC2       |dc1hana1  |    30001 |
1 |DC1       |YES           |SYNC        |ACTIVE      |              |        True |
|RH1      |dc2hana1 |30007 |xsengine     |        2 |      2 |DC2       |dc1hana1  |    30007 |        1
|DC1       |YES           |SYNC        |ACTIVE      |              |        True |
|RH1      |dc2hana1 |30003 |indexserver  |        3 |      2 |DC2       |dc1hana1  |    30003 |        1
|DC1       |YES           |SYNC        |ACTIVE      |              |        True |

status system replication site "1": ACTIVE
overall system replication status: ACTIVE

Local System Replication State
~~~~~~~~~~

mode: PRIMARY
site id: 2
site name: DC2
```

CHAPTER 10. TROUBLESHOOTING

10.1. THE SRHOOK CLUSTER ATTRIBUTE VALUE IS INCORRECT

When the **srHook** attribute value does not match the actual HANA system replication status, it can lead to unexpected behavior in the cluster when a failure of a primary instance occurs.

Check and correct your sudo configuration when the **srHook** attribute of the secondary site and the HANA system replication status do not match:

- The **srHook** cluster attribute of the secondary is empty.
- The **srHook** cluster attribute of the secondary is set to **SOK** while the HANA system replication is not healthy.
- The **srHook** cluster attribute of the secondary is set to **SFAIL** while the system replication is in **ACTIVE** state.

The primary site receives the events of HANA system replication changes and stores the result as a cluster attribute for the secondary site.

Procedure

1. Check for **crm_attribute** update errors in the **secure** log, since the command is executed using **sudo**. The log shows the command that the hook script tries to execute, but potentially fails. Check on the primary instance node for an error like **command not allowed**, like in this example:

```
[root]# grep crm_attribute /var/log/secure
... rh1adm : command not allowed ; PWD=/hana/shared/RH1/HDB02/<node> ; USER=root ;
COMMAND=/usr/sbin/crm_attribute -n hana_rh1_site_srHook_DC2 -v SFAIL -t crm_config -s
SAPHanaSR
```

2. Compare the logged **COMMAND** to your **sudoers** configuration. Check thoroughly and fix the **sudoers** file, so that you have a sudo entry that matches the command. As a temporary measure you can ensure that the sudo entry as such works by simplifying it with a wildcard to exclude typos in the command parameters as the cause:

```
[root]# cat /etc/sudoers.d/20-saphana
Defaults:<sid>adm !requiretty
<sid>adm ALL=(ALL) NOPASSWD: /usr/sbin/crm_attribute *
```

   - Replace **<sid>** with your lower-case HANA SID.

3. Verify that the command path is correct:

```
[root]# ls /usr/sbin/crm_attribute
/usr/sbin/crm_attribute
```

4. Fix the sudo configuration. For more information, see Configuring the HanaSR HA/DR provider for the srConnectionChanged() hook method.

5. Repeat any fixing steps on all nodes. The sudo configuration must be identical on all instances.

Additional resources

sudo(8) and **visudo(8)** man pages on your system

10.2. THE HANA INSTANCE DOES NOT START AFTER HOOK CHANGES

You recently made changes in the **global.ini** in a HA/DR provider section and the HANA instance does not start anymore.

Procedure

1. Go to the HANA trace logs directory, as the **<sid>adm** user:

```
rh1adm $ cdtrace
```

2. Check for errors related to the HA/DR providers in the HANA nameserver process alert log:

```
rh1adm $ grep ha_dr_provider nameserver_alert_*.trc
... ha_dr_provider   PythonProxyImpl.cpp(00145) : import of hanasr failed: No module named
'hanasr'
... ha_dr_provider   HADRProviderManager.cpp(00100) : could not load HA/DR Provider
'hanasr' from /usr/share/sap-hana-ha/
```

3. Identify the root cause, for example a misspelled HA/DR **provider** name or a wrong**path**. Check the path and the hook script name. In this example the HA/DR provider name **hanasr** is not matching the hook script name **HanaSR**:

```
rh1adm $ ls /usr/share/sap-hana-ha/
ChkSrv.py  HanaSR.py  samples
```

4. Correct the **HanaSR** HA/DR provider configuration:

```
[ha_dr_provider_hanasr]
provider = HanaSR
path = /usr/share/sap-hana-ha/
execution_order = 1
```

- **provider** must match the name of the Python hook script. It is case-sensitive without the **.py** file suffix.
- **path** must be the path in which the hook script is stored.

10.3. A CLUSTER NODE IS REPORTED AS OFFLINE DURING MAINTENANCE

When **maintenance-mode** is set for the cluster, for example, for a HANA update, it can still notice issues between the nodes, but does not trigger recovery actions yet.

If you encounter such a situation, you must first fix the cause of the issue before you lift the maintenance mode.

Example: the corosync communication between the nodes is blocked in a 2-node cluster

Both nodes report the other node as **offline**. If the **maintenance** mode is removed in this situation, the cluster tries to recover by fencing one node. This can have a severe impact on your ongoing HANA maintenance activity.

```
...
              *** Resource management is DISABLED ***
  The cluster will not attempt to start, stop or recover services

Node List:
  * Node hana1 (1): online, feature set 3.19.0
  * Node hana2 (2): UNCLEAN (offline)

Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02] (maintenance):
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started hana2
(UNCLEAN, maintenance)
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started hana1
(maintenance)
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02] (promotable,
maintenance):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Unpromoted hana2
(UNCLEAN, maintenance)
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Promoted hana1
(maintenance)
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02] (maintenance):
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started hana2
(UNCLEAN, maintenance)
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started hana1
(maintenance)
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):        Started hana1 (maintenance)
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):        Started hana2 (UNCLEAN,
maintenance)

...
```

Identify the root cause of the issue, for example:

- Planned network maintenance on the cluster communication connection in parallel to your HANA maintenance.
- Unplanned outage of network connections due to network device failures or misconfiguration on operating system or network level.
- Firewall configuration blocking cluster communication ports.

Fix any issue to prevent the cluster from taking recovery measures when the cluster maintenance is removed.

APPENDIX A. COMPONENT OPTIONS

A.1. HA/DR PROVIDER OPTIONS FOR HANASR

Parameters that are available for the configuration of the **HanaSR** HA/DR provider are shown below:

Provider options	Required	Default	Description
provider = HanaSR	yes		The provider parameter must be set to the hook script name without the **.py** ending.
path = /usr/share/sap-hana-ha/	yes		The full path to the location of the hook script.
execution_order = 1	yes		Set to **1** to ensure that the **HanaSR** HA/DR provider is always executed before other optional hook scripts when an event occurs.

A.2. HA/DR PROVIDER OPTIONS FOR CHKSRV

Parameters that are available for the configuration of the **ChkSrv** HA/DR provider are shown below:

Provider options	Required	Default	Description
provider = ChkSrv	yes		The provider parameter must be set to the hook script name without the **.py** ending.
path = /usr/share/sap-hana-ha/	yes		The full path to the location of the hook script.
execution_order = 2	yes		Set to **2** or higher. The mandatory **HanaSR** provider must always be run first.

Provider options	Required	Default	Description
action_on_lost	no	ignore	Action to be triggered when a lost indexserver is identified. • **ignore**: do nothing, just write to trace files. • **stop**: execute **sapcontrol … StopSystem**. • **kill**: execute **HDB kill-<signal>**. The signal can be defined by the separate parameter **kill_signal**. • **fence**: trigger the cluster to fence the node on which the indexserver failed. This option uses the **SAPHanaSR-hookHelper** tool and requires a sudo entry for the tool for the **<sid>adm** user.
kill_signal	no	9	The signal that is used with the **kill** action.
stop_timeout	no	20s	How many seconds to wait for the **stop** action to return. This value must be greater than the value of the HANA parameter **forcedterminationtimeout**.

A.3. SAPHANATOPOLOGY RESOURCE PARAMETERS

Parameters that are available for the configuration of **SAPHanaTopology** resources are shown below:

Resource options	Required	Default	Description
SID	yes		SAP system identifier.
InstanceNumber	yes		Number of the SAP HANA instance.
HANA_CALL_TIMEOUT	no	120	Defines the timeout - how long a call to HANA to receive information can take, for example, when the resource agent executes **landscapeHostConfiguration.py**. There are some specific calls to HANA which have their own timeout values. If you increase the timeout for HANA calls of this resource you must also consider increasing the operation timeout values of the same resource.

A.4. SAPHANACONTROLLER RESOURCE PARAMETERS

Parameters that are available for the configuration of **SAPHanaController** resources are shown below:

Resource options	Required	Default	Description
SID	yes		SAP system identifier.
InstanceNumber	yes		Number of the SAP HANA instance.
DIR_EXECUTABLE	no		The fully qualified path to binaries such as **sapstartsrv** and **sapcontrol**. Specify this parameter if you have changed the SAP kernel directory location after the default SAP installation. SAP standard paths are searched by default.

Resource options	Required	Default	Description
DIR_PROFILE	no		The fully qualified path to the SAP START profile. Specify this parameter if you have changed the SAP profile directory location after the default SAP installation. SAP standard paths are searched by default.
HANA_CALL_TIMEOUT	no	120	Defines the timeout - how long a call to HANA to receive information can take, for example, when the resource agent runs **landscapeHostConfiguration.py**. There are some specific calls to HANA which have their own timeout values. If you increase the timeout for HANA calls of this resource you must also consider increasing the operation timeout values of the same resource.
INSTANCE_PROFILE	no		The name of the SAP HANA instance profile. Specify this parameter if you have changed the name of the SAP HANA instance profile after the default SAP installation. SAP standard paths are searched by default.

Resource options	Required	Default	Description
PREFER_SITE_TAKEOVER	no	false	Defines whether the resource agent should prefer to trigger takeover to the secondary site instead of restarting the primary site locally. However, a takeover is only triggered if the SAP HANA landscape status is on **ERROR**. For **FATAL** a local restart is initiated. ● **true**: Prefer to trigger the takeover to the secondary site. ● **false**: Prefer to restart locally.
AUTOMATED_REGISTER	no	false	Defines whether the resource agent automatically registers a former primary instance as a secondary during cluster resource start and if the **DUPLICATE_PRIMARY_TIMEOUT** condition is met. Registering an instance as a secondary initiates a data synchronisation from the primary and might overwrite local data. ● **false**: Manual intervention is required. ● **true**: The former primary is automatically registered as a secondary.

Resource options	Required	Default	Description
DUPLICATE_PRIMARY_TIMEOUT	no	7200	The time difference required between two primary time stamps (LPTs), in case a dual-primary situation occurs. If the difference between both nodes' last primary time stamps is less than **DUPLICATE_PRIMARY_TIMEOUT**, then the cluster holds one or both instances in a **WAITING** status. This gives an admin the chance to react to a failover. How the recovery proceeds after the **DUPLICATE_PRIMARY_TIMEOUT** has passed depends on the parameter **AUTOMATED_REGISTER**.

Resource options	Required	Default	Description
ON_FAIL_ACTION	no	proceed	Set this parameter to **fence** to let the resource trigger the node to be fenced. This functionality uses the **SAPHanaSR-hookHelper** tool to run the fence action. The **fence** option can improve the takeover and recovery time of a HANA instance if the instance stop usually takes a longer time. • **proceed**: continue with the usual action to a failed resource. • **fence**: react with a resource stop failure to trigger the cluster fencing action, but only if the **srHook** status is **SOK**. If the **srHook** status is **SFAIL**, the local recovery takes place instead, and fencing is not triggered.

We recommend that you set **PREFER_SITE_TAKEOVER** to **true**. This allows the HA cluster to trigger a takeover when a failure of the primary HANA instance is detected. In most cases it takes less time for the new HANA primary instance to become fully active after a takeover, than it takes for the original primary instance to restart and reload all data back from disk into memory.

Leave **AUTOMATED_REGISTER** set to **false** to give the operator the option to first verify the health and data consistency of the previously failed primary instance, before you manually register it as the new secondary instance to re-establish the HANA system replication between both instances and manually start the instance.

Set **AUTOMATED_REGISTER** to **true** to enable the automatic registration of the former primary instance as the new secondary after a takeover occurs. This increases the availability of the HANA system replication setup and prevents so-called dual-primary situations in the SAP HANA system replication environment.

A.5. SAPHANAFILESYSTEM RESOURCE PARAMETERS

Parameters that are available for the configuration of **SAPHanaFilesystem** resources are shown below:

Resource options	Required	Default	Description
SID	yes		SAP system identifier.
InstanceNumber	yes		Number of the SAP HANA instance.
DIRECTORY	no	**/hana/shared/<SID>/**	Path to the directory to be monitored. The resource agent creates data in its own subdirectory **.heartbeat_SAPHanaFilesystem** inside the **DIRECTORY** path. Do not change this hidden subdirectory.
ON_FAIL_ACTION	no	fence	Action in case of a repeated monitor failure. • **ignore**: Do nothing, just report the failure into logs. • **fence**: Trigger stop failure and node fencing, if the required conditions are met.

APPENDIX B. USEFUL INFORMATION

B.1. EXPLAINING CLUSTER INFORMATION

When the cluster resources start they execute the first monitor operation and gather initial resource information. The HANA resources add node attributes and cluster properties for the collected landscape information, which describes the current state of SAP HANA databases on the cluster nodes.

Scale-up cluster status with node attributes

```
[root]# pcs status --full
...

Node List:
  * Node node1 (1): online, feature set 3.19.0
  * Node node2 (2): online, feature set 3.19.0

Full List of Resources:
  * rsc_fence       (stonith:<fence agent>):     Started node1
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]:
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node2
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node1
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02] (promotable):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted node2
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Promoted node1
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02]:
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started node2
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started node1
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):        Started node1
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):        Started node2

Node Attributes:
  * Node: node1 (1):
    * hana_rh1_clone_state              : PROMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC1
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.078.00
    * hana_rh1_vhost                    : node1
    * master-rsc_SAPHanaCon_RH1_HDB02   : 150
  * Node: node2 (2):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC2
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.078.00
    * hana_rh1_vhost                    : node2
    * master-rsc_SAPHanaCon_RH1_HDB02   : 100

...
```

Scale-out cluster status with node attributes

```
[root]# pcs status --full
...

Node List:
  * Node dc1hana1 (1): online, feature set 3.19.6
  * Node dc1hana2 (2): online, feature set 3.19.6
  * Node dc2hana1 (3): online, feature set 3.19.6
  * Node dc2hana2 (4): online, feature set 3.19.6

Full List of Resources:
  * rsc_fence       (stonith:<fence agent>):     Started dc1hana1
  * rsc_fence_aws       (stonith:fence_aws):     Started dc1hana1
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]:
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):         Started dc1hana2
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):         Started dc1hana1
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):         Started dc2hana1
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):         Started dc2hana2
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02] (promotable):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Unpromoted dc1hana2
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Promoted dc1hana1
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Unpromoted dc2hana1
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Unpromoted dc2hana2
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02]:
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started dc1hana2
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started dc1hana1
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started dc2hana1
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started dc2hana2
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPAddr2):         Started dc1hana1
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPAddr2):         Started dc2hana1

Node Attributes:
  * Node: dc1hana1 (1):
    * hana_rh1_clone_state              : PROMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC1
    * hana_rh1_sra                      : -
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc1hana1
    * master-rsc_SAPHanaCon_RH1_HDB02   : 150
  * Node: dc1hana2 (2):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : slave:slave:worker:slave
    * hana_rh1_site                     : DC1
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc1hana2
    * master-rsc_SAPHanaCon_RH1_HDB02   : -10000
  * Node: dc2hana1 (3):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC2
    * hana_rh1_sra                      : -
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc2hana1
```

```
    * master-rsc_SAPHanaCon_RH1_HDB02   : 100
  * Node: dc2hana2 (4):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : slave:slave:worker:slave
    * hana_rh1_site                     : DC2
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.083.00
    * hana_rh1_vhost                    : dc2hana2
    * master-rsc_SAPHanaCon_RH1_HDB02   : -12200

...
```

Cluster properties

The new generation of SAP HANA resource agents stores information about the HANA instance as cluster properties in the property set named **SAPHanaSR**.

You can query the CIB (cluster information base) as the **root** user to check the content of the cluster attributes. The HANA resources and the **HanaSR** hook update these attributes.

Example of a scale-up cluster:

```
[root]# cibadmin --query --xpath "//crm_config//cluster_property_set[@id='SAPHanaSR']"
<cluster_property_set id="SAPHanaSR">
  <nvpair id="SAPHanaSR-hana_rh1_site_srHook_DC2" name="hana_rh1_site_srHook_DC2"
value="SOK"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_mns_DC2" name="hana_rh1_site_mns_DC2"
value="node2"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_mns_DC1" name="hana_rh1_site_mns_DC1"
value="node1"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lpt_DC2" name="hana_rh1_site_lpt_DC2" value="30"/>
  <nvpair id="SAPHanaSR-hana_rh1_glob_sec" name="hana_rh1_glob_sec" value="DC2"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_opMode_DC2" name="hana_rh1_site_opMode_DC2"
value="logreplay"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srHook_DC1" name="hana_rh1_site_srHook_DC1"
value="PRIM"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lpt_DC1" name="hana_rh1_site_lpt_DC1"
value="1746780482"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srPoll_" name="hana_rh1_site_srPoll_" value="SOK"/>
  <nvpair id="SAPHanaSR-hana_rh1_glob_topology" name="hana_rh1_glob_topology"
value="ScaleUp"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lss_DC2" name="hana_rh1_site_lss_DC2" value="4"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lss_DC1" name="hana_rh1_site_lss_DC1" value="4"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srr_DC2" name="hana_rh1_site_srr_DC2" value="S"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srr_DC1" name="hana_rh1_site_srr_DC1" value="P"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srMode_DC2" name="hana_rh1_site_srMode_DC2"
value="sync"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srMode_" name="hana_rh1_site_srMode_" value="sync"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srPoll_DC2" name="hana_rh1_site_srPoll_DC2"
value="SOK"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srMode_DC1" name="hana_rh1_site_srMode_DC1"
value="sync"/>
  <nvpair id="SAPHanaSR-hana_rh1_glob_prim" name="hana_rh1_glob_prim" value="DC1"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srPoll_DC1" name="hana_rh1_site_srPoll_DC1"
```

```
value="PRIM"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_opMode_DC1" name="hana_rh1_site_opMode_DC1"
value="logreplay"/>
</cluster_property_set>
```

Example of a scale-out cluster:

```
[root]# cibadmin --query --xpath "//crm_config//cluster_property_set[@id='SAPHanaSR']"
<cluster_property_set id="SAPHanaSR">
  <nvpair id="SAPHanaSR-hana_rh1_site_mns_DC2" name="hana_rh1_site_mns_DC2"
value="dc2hana1"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_mns_DC1" name="hana_rh1_site_mns_DC1"
value="dc1hana1"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lpt_DC2" name="hana_rh1_site_lpt_DC2" value="30"/>
  <nvpair id="SAPHanaSR-hana_rh1_glob_sec" name="hana_rh1_glob_sec" value="DC2"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_opMode_DC2" name="hana_rh1_site_opMode_DC2"
value="logreplay"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srHook_DC1" name="hana_rh1_site_srHook_DC1"
value="PRIM"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lpt_DC1" name="hana_rh1_site_lpt_DC1"
value="1763549830"/>
  <nvpair id="SAPHanaSR-hana_rh1_glob_topology" name="hana_rh1_glob_topology"
value="ScaleOut"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lss_DC2" name="hana_rh1_site_lss_DC2" value="4"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lss_DC1" name="hana_rh1_site_lss_DC1" value="4"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srr_DC2" name="hana_rh1_site_srr_DC2" value="S"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srr_DC1" name="hana_rh1_site_srr_DC1" value="P"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srMode_DC2" name="hana_rh1_site_srMode_DC2"
value="sync"/>
  <nvpair id="SAPHanaSR-hana_rh1_glob_prim" name="hana_rh1_glob_prim" value="DC1"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srPoll_DC1" name="hana_rh1_site_srPoll_DC1"
value="PRIM"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srPoll_DC2" name="hana_rh1_site_srPoll_DC2"
value="SOK"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srMode_DC1" name="hana_rh1_site_srMode_DC1"
value="sync"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_opMode_DC1" name="hana_rh1_site_opMode_DC1"
value="logreplay"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srHook_DC2" name="hana_rh1_site_srHook_DC2"
value="SOK"/>
</cluster_property_set>
```

SAPHanaSR-showAttr

You can use the tool SAPHanaSR-showAttr to display all of the HANA cluster attribute information in a preformatted overview.

Check this status in addition to pcs status [--full] to see the overall landscape health.

Example of a scale-up cluster:

```
[root]# SAPHanaSR-showAttr
Global cib-update dcid prim sec sid topology
---------------------------------------------
```

```
global 0.550.2    2    DC1  DC2 RH1 ScaleUp

Resource                promotable
------------------------------------
cln_SAPHanaCon_RH1_HDB02 true
cln_SAPHanaTop_RH1_HDB02

Site lpt        lss mns   opMode    srHook srMode srPoll srr
-------------------------------------------------------------
DC2  30         4   node2 logreplay SOK    sync   SOK    S
DC1  1746780164 4   node1 logreplay PRIM   sync   PRIM   P

Host  clone_state roles                      score site srah version     vhost
-----------------------------------------------------------------------------------
node1 PROMOTED    master1:master:worker:master 150   DC1  -    2.00.078.00 node1
node2 DEMOTED     master1:master:worker:master 100   DC2  -    2.00.078.00 node2
```

Example of a scale-out cluster:

```
[root]# SAPHanaSR-showAttr
Global cib-update dcid prim sec sid topology
---------------------------------------------
global 0.1955.0   1    DC1  DC2 RH1 ScaleOut

Resource                promotable
------------------------------------
cln_SAPHanaCon_RH1_HDB02 true
cln_SAPHanaTop_RH1_HDB02

Site lpt        lss mns      opMode    srHook srMode srPoll srr
-----------------------------------------------------------------
DC2  30         4   dc2hana1 logreplay SOK    sync   SOK    S
DC1  1756385739 4   dc1hana1 logreplay PRIM   sync   PRIM   P

Host     clone_state roles                      score  site sra srah version     vhost
-------------------------------------------------------------------------------------------
dc1hana1 PROMOTED    master1:master:worker:master 150    DC1  -   -    2.00.083.00 dc1hana1
dc1hana2 DEMOTED     slave:slave:worker:slave     -10000 DC1      -    2.00.083.00 dc1hana2
dc2hana1 DEMOTED     master1:master:worker:master 100    DC2  -   -    2.00.083.00 dc2hana1
dc2hana2 DEMOTED     slave:slave:worker:slave     -12200 DC2      -    2.00.083.00 dc2hana2
```